

AURIC ARTISAN · USER DOCUMENTATION

Services

The Auric Artisan services you can use from your own sites and code — the Font CDN, the Icon CDN and the public REST API.

Updated 2026-09-25 · auricartisan.com

Contents

Services

Font CDN

Font CDN documentation

- Getting started
- Embed a family
- Find a family
- Self-host a family
- Use the loader
- Reference
- Troubleshooting
- FAQ

Other material

- Licences
- Glossary
- Limits and accuracy
- Privacy
- Related

Icon CDN

Icon CDN documentation

- Getting started
- Collections
- Embed an icon
- Export and self-host
- Style, size and colour
- Use the loader
- Reference
- Troubleshooting
- FAQ

Other material

- Licences
- Glossary
- Limits and accuracy
- Privacy
- Related

Auric Artisan REST API

Public API documentation

- Getting started
- Authentication

Errors

Request examples

Plans, credits and quotas

SDKs and tooling

Reference

Troubleshooting

FAQ

Endpoints

Accessibility

Analyzer

Collections

Colour and colour science

Discovery endpoints

Palettes, harmonies, gradients and shades

Design tokens and personalisation

Usage, jobs, webhooks and GraphQL

Vision and Ishihara

Other material

Glossary

Limits and accuracy

Privacy

Release notes

Related

SERVICES

Services

The Auric Artisan services you can use from your own sites and code — the Font CDN, the Icon CDN and the public REST API.

Besides the website, the browser extension and the editor extensions, Auric Artisan offers services you use from your own projects: two content delivery networks that serve fonts and icons to your web pages, and a REST API that exposes its colour science and accessibility checks to your code.

Folders

Folder	What it covers
font-cdn/	fonts.auricartisan.com: the font families it serves, embedding them, the loader, self-hosting and licences
icon-cdn/	icons.auricartisan.com: the icon collections, the loader, embedding icons, and licences
public-api/	api.auricartisan.com: keys, authentication, plans and credits, every endpoint, errors and limits

SERVICES > FONT CDN

Font CDN

What the Auric Artisan Font CDN at fonts.auricartisan.com serves, who it is for, and how to put a font on your page in a few steps.

The Auric Artisan Font CDN, at <https://fonts.auricartisan.com>, serves open-licence font families to your web pages. Each family has its own stylesheet of `@font-face` rules that you link like a Google Fonts stylesheet, and a small loader script can load families declaratively or from JavaScript. The font files themselves are served from the same host, so a page that uses the CDN makes no request to any third party.

The catalogue holds 2,213 families and 4,883 font files, of which 550 families are variable fonts. Most are under the SIL Open Font License; others use Apache 2.0, public-domain dedications, MIT or other open licences, and every family's own licence text is available. A small part of the catalogue is tagged **Installed:** system fonts that are listed for reference but not served, because their licences do not allow redistribution.

You browse the catalogue, preview families in your own words, and copy embed code in the Font Library at <https://auricartisan.com/tool/font/>. That page is documented with the website tools; these pages cover the CDN itself: what it serves, how to embed a family, the loader, self-hosting and licences.

Who it is for

- **Web designers and developers** who want open-licence fonts without a third-party font service.
- **Teams with privacy or offline requirements** who prefer to self-host, using the downloadable kits.
- **Prototypers** who need to try many families quickly from JavaScript.

Where to find it

What	Where
CDN host	https://fonts.auricartisan.com
Loader script	https://fonts.auricartisan.com/font-cdn.js
Family stylesheets	<a href="https://fonts.auricartisan.com/css/<family-id>.css">https://fonts.auricartisan.com/css/<family-id>.css
Browse and copy embed code	https://auricartisan.com/tool/font/
Licence reference	https://fonts.auricartisan.com/LICENSE.md

Quick start

1. Open <https://auricartisan.com/tool/font/> and find a family, for example Alegreya.
2. Open the family and go to its **Use & Embed** tab.
3. Copy the **Link** snippet:

```
<link rel="stylesheet" href="https://fonts.auricartisan.com/css/alegreya.css">
```

4. Paste it into the `<head>` of your page.

5. Apply the family in your CSS:

```
.headline { font-family: 'Alegreya', serif; }
```

6. Check the family's **License** tab, and keep its terms.

7. To host the files yourself instead, choose **Download kit** and follow [Self-host a family](#).

What you can do

- Link a family's stylesheet, or `@import` it from your CSS.
- Load one or many families with a single script tag, or from JavaScript with `AuricFonts.load()`.
- Use variable fonts across their full weight range.
- Register a single font face from any URL with `AuricFonts.face()`.
- Search the catalogue and read a family's full metadata from JavaScript.
- Download a self-hosting kit with the font files, a stylesheet, a demo page and the licence.
- Copy a font stack with fallbacks, CSS variables, Tailwind, SCSS or design-token snippets from the Font Library.

In this folder

- docs/
 - [Getting started](#)
 - [Embed a family](#)
 - [Use the loader](#)
 - [Self-host a family](#)
 - [Find a family](#)
 - [Reference](#)
 - [Troubleshooting](#)
 - [FAQ](#)
- others/
 - [Glossary](#)
 - [Licences](#)
 - [Limits and accuracy](#)
 - [Related](#)
 - [Privacy](#)

SERVICES > FONT CDN

Font CDN documentation

The contents of the Font CDN documentation, from a first embed to the loader API and self-hosting.

These pages explain how to use fonts from fonts.auricartisan.com on your own pages: linking a family's stylesheet, the loader script and its JavaScript API, and downloading a kit to host the files yourself.

Page	What it covers
Getting started	How the CDN is organised, and a first family on a page
Embed a family	Link, <code>@import</code> , <code>@font-face</code> and usage snippets; weights, italics and variable fonts
Use the loader	The <code>font-cdn.js</code> script tag and the <code>AuricFonts</code> JavaScript API
Self-host a family	Download kits: what they contain and how to install them
Find a family	Finding a family and its id, on the Font Library page or from JavaScript
Reference	Every URL pattern, loader attribute, method, snippet and file
Troubleshooting	Fonts that do not appear, wrong weights, licences and more
FAQ	Short answers

SERVICES > FONT CDN

Getting started

How the Font CDN is organised — family ids, stylesheets and font files — and how to put a first family on a page.

How the CDN is organised

Everything is served from <https://fonts.auricartisan.com>:

Path	What it is
<code>/css/<family-id>.css</code>	One stylesheet per family, with an <code>@font-face</code> rule for each style
<code>/font/...</code>	The font files the stylesheets point to
<code>/font-cdn.js</code>	The loader script
<code>/data/index.json</code>	The catalogue: one short record per family
<code>/data/detail/<group>.json</code>	Full metadata for the families in a group (<code>A-F</code> , <code>G-L</code> , <code>M-R</code> , <code>S-Z</code>)
<code>/LICENSE.md</code>	The licence reference for the collection

You do not need to know the file paths inside `/font/`: the family stylesheet refers to them for you.

Family ids

Every family has an **id**, used in its stylesheet URL. The id is the family name in lower case, with apostrophes and quotation marks removed and every run of other characters replaced by a hyphen:

Family	Id	Stylesheet
Alegreya	alegreya	https://fonts.auricartisan.com/css/alegreya.css
Roboto Slab	roboto-slab	https://fonts.auricartisan.com/css/roboto-slab.css
Open Sans	open-sans	https://fonts.auricartisan.com/css/open-sans.css

The Font Library's **Use & Embed** tab always shows the exact URL for a family.

Your first family

1. Add the family's stylesheet to the `<head>` of your page:

```
<link rel="stylesheet" href="https://fonts.auricartisan.com/css/alegreya.css">
```

2. Use the family in your CSS, with a fallback:

```
body { font-family: 'Alegreya', serif; }
```

3. Load the page. The text appears in Alegreya once the font file has downloaded. Until then the browser shows the fallback, because the stylesheets use `font-display: swap`.

What a family stylesheet contains

Each stylesheet starts with a comment naming the family, its licence and copyright, and how many files and weights it has, then one `@font-face` rule per file:

```

/* Alegreya – Auric Artisan Font CDN
 * SIL Open Font License 1.1 · Copyright 2011 The Alegreya Project Authors
 (https://github.com/huertatipografica/Alegreya)
 * 2 file(s), weights 400 500 600 700 800 900, variable
 */
@font-face {
  font-family: 'Alegreya';
  font-style: italic;
  font-weight: 400 900;
  font-display: swap;
  src: url('../font/A-F/ALEGREYA/Alegreya-Italic%5Bwght%5D.ttf') format('truetype-variations');
}
@font-face {
  font-family: 'Alegreya';
  font-style: normal;
  font-weight: 400 900;
  font-display: swap;
  src: url('../font/A-F/ALEGREYA/Alegreya%5Bwght%5D.ttf') format('truetype-variations');
}

```

A variable family, like this one, covers a whole weight range with one file per style. A static family has one rule per weight and style.

Next steps

- [Embed a family](#) for the other ways to embed.
- [Use the loader](#) to load several families with one tag, or from JavaScript.
- [Self-host a family](#) to serve the files from your own site.

SERVICES > FONT CDN

Embed a family

Every way to embed a Font CDN family — link tag, CSS @import, @font-face rules and usage CSS — plus weights, italics, variable axes and several families at once.

The Font Library's **Use & Embed** tab gives each family six ready snippets: **Link**, **Import**, **Script**, **JS**, **@font-face** and **Usage**. Choose one, then **Copy**. The tab also shows the family's weights as chips under **Selected weights**, and a note naming the licence. This page explains each snippet.

Link (recommended)

Add the stylesheet to your page's `<head>`:

```

<!-- Self-hosted stylesheet (Google-Fonts style) -->
<link rel="stylesheet" href="https://fonts.auricartisan.com/css/alegreya.css">

```

Put it before your own stylesheets so your CSS can use the family straight away.

Import

From inside a CSS file:

```
/* In your CSS file */
@import url('https://fonts.auricartisan.com/css/alegreya.css');
```

An `@import` must come before other rules in the file. A link tag is usually faster, because the browser finds it sooner.

Script and JS

The loader script adds the stylesheet for you:

```
<!-- Auto-load with the CDN script -->
<script src="https://fonts.auricartisan.com/font-cdn.js"
        data-families="Alegreya"></script>
```

```
// Programmatic load
AuricFonts.load('Alegreya').then(() => {
  document.body.style.fontFamily = "'Alegreya', serif";
});
```

See [Use the loader](#).

@font-face

The `@font-face` snippet gives the rules themselves, pointing at the font files on the CDN, for up to four of the family's styles. Use it when you want to control the rules, for example to rename the family or include only some styles:

```
@font-face {
  font-family: 'Alegreya';
  font-style: normal;
  font-weight: 400 900;
  font-display: swap;
  src: url('https://fonts.auricartisan.com/font/A-F/ALEGREYA/Alegreya%5Bwght%5D.ttf') format('truetype-variations');
}
```

Variable styles are written with a weight range, and with a `font-stretch` range when the font has a width axis.

Usage

The **Usage** snippet applies the family with a fallback for its category:

```
/* Apply the family */
.headline {
  font-family: 'Alegreya', serif;
  font-weight: 400 900; /* variable range */
}
```

For a static family the snippet uses `font-weight: 400;`. The fallbacks used by category are:

Category	Fallback in the Usage snippet
Sans-serif	<code>sans-serif</code>
Serif	<code>serif</code>
Monospace	<code>monospace</code>
Display	<code>system-ui</code>
Handwriting	<code>cursive</code>

The Font Library's **Develop** tab offers a fuller font stack with safe fallbacks; see [Reference](#).

Weights and italics

- A family's stylesheet declares **every** style the CDN has for it. You do not choose weights in the URL.
- Set the weight you want in your CSS with `font-weight`. If the family lacks that weight, the browser uses the nearest one it has, or synthesises bold.
- Use `font-style: italic` for italics, if the family has them. The **Styles** tab of the Font Library lists each weight and style.

Variable fonts

A variable font holds a continuous range of a design axis in one file. On the CDN, 550 families are variable. Their stylesheets declare the range, such as `font-weight: 400 900`, so any weight in between works:

```
h1 { font-family: 'Alegreya', serif; font-weight: 650; }
```

Other axes, such as width (`wdth`), slant (`slnt`) or optical size (`opsz`), can be set with standard CSS such as `font-stretch` or `font-variation-settings`, where the family has them. The Font Library's **Specimen** tab has sliders for each axis a family provides.

Several families at once

Link each stylesheet:

```
<link rel="stylesheet" href="https://fonts.auricartisan.com/css/inter.css">
<link rel="stylesheet" href="https://fonts.auricartisan.com/css/lora.css">
```

Or use one script tag with the family names separated by `|`:

```
<script src="https://fonts.auricartisan.com/font-cdn.js"
  data-families="Inter|Lora"></script>
```

In the Font Library, add families to your selection, then choose **Embed** in the **Your kit** tray to get **Combined embed code** for all of them: the link tags, the equivalent script tag, and a `:root` block of `--font-<id>` variables.

Related pages

- [Use the loader](#)
- [Self-host a family](#)
- [Licences](#)

SERVICES > FONT CDN

Find a family

Find a family and its id for the Font CDN, on the Font Library page or from JavaScript, and read what the catalogue records about it.

On the Font Library page

The Font Library at <https://auricartisan.com/tool/font/> is the catalogue's front end. It renders every family in itself, lets you set your own words in it, and gives the embed code and kit for the ones you keep. Its main controls are:

- **Search fonts** ("Family, designer or foundry");
- sorting by Alphabetical, Accessibility score, UI score, Readability score, Most weights, Most glyphs, Variable first, or Surprise me;
- category tabs and filters;
- **Recommend**, a recommender for a brief;
- **Collections**, curated sets;
- **Selected**, the families you have gathered for a combined embed or kit.

Each family opens with tabs including **Specimen**, **Suitability**, **Pairings**, **Glyphs**, **Features**, **Styles**, **About**, **License**, **Charset**, **Develop** and **Use & Embed**. The page itself is documented with the website tools; see [Related](#).

From JavaScript

The loader can search the catalogue:

```
const hits = await AuricFonts.search('mono'); // name, designer or category
const all  = await AuricFonts.list();
```

Each record in the catalogue includes, among other fields, the family's **id**, **name**, **group**, **category**, **weights**, whether it is **variable** and its **axes**, whether it has italics, whether it is monospace, its **license** key, **designer**, the scripts it covers and its character count.

For everything about one family, use **get()**:

```
const meta = await AuricFonts.get('Alegreya'); // or the id, 'alegreya'
```

It returns the family's full metadata, including its styles and files, or **null** if there is no such family.

Categories

Families are sorted into five categories: sans-serif, serif, display, handwriting and monospace. Categories are worked out from the fonts' own data and outlines, so a few families may sit in a category you would not choose; see [Limits and accuracy](#).

Related pages

- [Use the loader](#)
- [Reference](#)

SERVICES > FONT CDN

Self-host a family

Download a font kit from the Font Library, understand what it contains, and serve the family from your own site.

Instead of loading fonts from fonts.auricartisan.com, you can download a family and serve it from your own site. The Font Library builds a ready-to-use kit in your browser.

Download a kit

For one family:

1. Open the family in the Font Library at <https://auricartisan.com/tool/font/>.
2. Choose **Download kit** ("Download N bundled font files with CSS and license notes"). The same button is in the **Styles** tab under **Download self-hosting kit**.
3. Your browser saves `<Family>-font-kit.zip`.

For several families:

1. Add them to your selection. They collect in the **Your kit** tray.
2. Choose **Download** in the tray (it reads **Download selected (N)**). The file is named `auric-font-selection-kit.zip` (or after the family when there is one).

The Font Library shows "Preparing download kit..." while it gathers the files, then "Downloaded" and the file name.

What a kit contains

For each family, a folder with:

File	What it is
<code>fonts/...</code>	Every font file the CDN serves for the family
<code>css/<family-id>.css</code>	<code>@font-face</code> rules pointing at <code>../fonts/</code> , and a <code>--font-<family-id></code> variable with a fallback
<code>demo.html</code>	A page that shows the family, using the stylesheet
<code>README.txt</code>	The family, category, designer, licence, file list and three steps to use it
<code>LICENSE.txt</code>	The family's licence text, when the library has it

If a file could not be fetched, the kit adds `DOWNLOAD_NOTES.txt` listing what is missing.

Install it on your site

1. Unzip the kit and copy the family's folder into your site, for example to `/fonts/alegreya/`.
2. Keep the `fonts`, `css` and licence files together.
3. Link the stylesheet from your pages:

```
<link rel="stylesheet" href="/fonts/alegreya/css/alegreya.css">
```

4. Apply the family:

```
body { font-family: 'Alegreya', serif; }
```

or use the variable the stylesheet defines, such as `var(--font-alegreya)`.

5. Open `demo.html` from the kit to check the files work.

Point the loader at your copy

If you use `font-cdn.js`, you can host it with your own stylesheets and data and set its base address; see [Use the loader](#).

Families that cannot be downloaded

Families tagged **Installed** are system fonts: the library lists them but does not serve their files, because their licences do not allow redistribution. For these, the download button reads **Installed only**, and the Font Library says "Only bundled library fonts can be downloaded".

Keep the licence

Every kit includes the licence information for a reason: most open font licences require the licence to travel with the files. See [Licences](#).

Related pages

- [Embed a family](#)
- [Licences](#)

SERVICES > FONT CDN

Use the loader

Load Font CDN families with the `font-cdn.js` script tag or its AuricFonts JavaScript API, and configure where it loads from.

`font-cdn.js` is a small script with no dependencies. It adds family stylesheets to your page, waits until fonts are ready, registers single font faces, and lets you search the catalogue. It creates a global object, `AuricFonts` (also available as `AuricFontLoader`).

Declarative: one script tag

```
<script src="https://fonts.auricartisan.com/font-cdn.js"
  data-families="Alegreya|Roboto Slab|Open Sans:wght@400;700"></script>
```

Attribute	What it does
<code>data-families</code>	The families to load, separated by a vertical bar; each can carry a weight list (see Family specs)
<code>data-fonts</code>	Another name for <code>data-families</code>
<code>data-display</code>	The <code>font-display</code> value for faces added with <code>AuricFonts.face()</code> . Default <code>swap</code> .
<code>data-base</code>	Load stylesheets and data from another address; see Where it loads from

For each family, the loader adds a `<link rel="stylesheet">` for its CDN stylesheet to the page's `<head>`, once.

Programmatic: AuricFonts

Include the script without `data-families`, then call the API:

```
<script src="https://fonts.auricartisan.com/font-cdn.js"></script>
<script>
  AuricFonts.load('Alegreya').then(() => {
    document.body.style.fontFamily = "'Alegreya', serif";
  });
</script>
```

Load families

```
await AuricFonts.load('Alegreya'); // one family
await AuricFonts.load(['Inter:400,700', 'Lora']); // several at once
```

`load()` adds the family's stylesheet and resolves, with `{ id, family }` (an array of them for several), once the browser reports the requested weights are ready. Use it to avoid a flash of the fallback font before you measure or draw text.

Register one face from any file

```
AuricFonts.face({ family: 'Demo', url: 'https://example.com/Foo.ttf', weight: 400 });
```

Option	Default	Meaning
<code>family</code>	—	The family name to register
<code>url</code>	—	The font file's URL
<code>local</code>	—	A locally installed font name to try first
<code>weight</code>	<code>400</code>	The weight
<code>style</code>	<code>normal</code>	<code>normal</code> or <code>italic</code>

Option	Default	Meaning
<code>display</code>	<code>swap</code> (or <code>data-display</code>)	The <code>font-display</code> value
<code>stretch</code>	—	The <code>font-stretch</code> value

It resolves with the family name, and does not reject if the file fails to load.

Other methods

Method	Returns	Use
<code>AuricFonts.cssUrl(spec)</code> or <code>AuricFonts.css(spec)</code>	The stylesheet URL for a family	Build your own link or <code>@import</code>
<code>AuricFonts.injectCss(cssText, id)</code>	The added <code><style></code> element	Add raw <code>@font-face</code> or <code>@import</code> CSS
<code>AuricFonts.whenReady(family, weights)</code>	A promise	Wait for a family you added another way
<code>AuricFonts.list()</code>	A promise of every family record	Build your own picker
<code>AuricFonts.search(query)</code>	A promise of matching family records	Match on name, designer or category
<code>AuricFonts.get(idOrName)</code>	A promise of the family's full metadata, or <code>null</code>	Read styles, axes, licence and more
<code>AuricFonts.slugify(name)</code>	The family id	Turn a name into its id
<code>AuricFonts.base</code>	The address it loads from	Read or change it
<code>AuricFonts.index</code>	The catalogue, once loaded	Inspect what <code>list()</code> fetched

```
const mono = await AuricFonts.search('mono');
const meta = await AuricFonts.get('alegreya');
```

`list()`, `search()` and `get()` download catalogue data from the CDN the first time you call them: the index is large, so call them only when you need them.

Family specs

Anywhere the loader takes a family, you can write:

Spec	Meaning
<code>Alegreya</code>	The whole family
<code>Open Sans:wght@400;700</code>	The family, waiting for weights 400 and 700
<code>Roboto:400,700i</code>	Weights 400 and 700; the <code>i</code> marks italics

Spec	Meaning
<code>{ family: 'Inter', weights: [400, 700] }</code>	The same as an object; <code>id</code> can be given instead of <code>family</code>

The weights tell the loader what to wait for. The stylesheet it adds always declares the whole family.

Where it loads from

By default the loader loads stylesheets and data from the address it was itself loaded from. To point it elsewhere, for example at a self-hosted copy, set one of these (the first found wins):

1. `data-base` on the script tag;
2. `window.AURIC_FONT_BASE` (or `window.AURIC_FONT_ASSET_BASE`), set before the script loads;
3. `<meta name="auric-font-base" content="...">` in the page.

You can also set `AuricFonts.base` from JavaScript before calling `load()`.

Related pages

- [Embed a family](#)
- [Reference](#)
- [Privacy](#)

SERVICES > FONT CDN

Reference

Every URL pattern, file, loader attribute, method and snippet of the Auric Artisan Font CDN.

Catalogue

Fact	Value
Host	https://fonts.auricartisan.com
Families catalogued	2,213
Font files	4,883
Variable families	550
Families with italics	401
Served from the library	2,023 families
Listed as Installed (not served)	190 families
Categories	Sans-serif 780, serif 1,172, display 81, handwriting 97, monospace 83

Fact	Value
Groups	A-F , G-L , M-R , S-Z
Catalogue built	2 June 2026

These figures come from the catalogue's own statistics, served at `/data/stats.json` and `/data/meta.json`.

URL patterns

Pattern	What it serves
<code>/css/<family-id>.css</code>	The family's stylesheet: a header comment and one <code>@font-face</code> rule per file
<code>/font/<group>/<FOLDER>/<file></code>	Font files, referenced by the stylesheets
<code>/font-cdn.js</code>	The loader script
<code>/data/index.json</code>	One record per family
<code>/data/detail/<group>.json</code>	Full metadata for the families in a group
<code>/data/stats.json</code> , <code>/data/meta.json</code>	Catalogue statistics and build information
<code>/LICENSE.md</code>	The collection's licence reference

The family id is the name in lower case, with quotation marks and apostrophes removed and other characters replaced by hyphens (`Roboto Slab` becomes `roboto-slab`).

Stylesheet rules

Property	Value
<code>font-family</code>	The family name
<code>font-style</code>	<code>normal</code> or <code>italic</code>
<code>font-weight</code>	One weight, or a range for variable fonts (for example <code>400 900</code>)
<code>font-stretch</code>	A percentage range, for variable fonts with a width axis
<code>font-display</code>	<code>swap</code>
<code>src</code>	The file URL with its format (for example <code>truetype-variations</code>)

Loader: script attributes

Attribute	Meaning	Default
<code>data-families</code>	Families to load, separated by a vertical bar	—

Attribute	Meaning	Default
<code>data-fonts</code>	Alias for <code>data-families</code>	—
<code>data-display</code>	<code>font-display</code> for faces added with <code>face()</code>	<code>swap</code>
<code>data-base</code>	The address to load stylesheets and data from	The script's own address

Other ways to set the base address: `window.AURIC_FONT_BASE`, `window.AURIC_FONT_ASSET_BASE`, or `<meta name="auric-font-base">`.

Loader: AuricFonts API

The global `AuricFonts` (alias `AuricFontLoader`).

Member	Parameters	Returns
<code>load(spec)</code>	A spec or an array of specs	Promise of <code>{ id, family }</code> (or an array) when ready
<code>face(options)</code>	<code>family</code> , <code>url</code> , <code>local</code> , <code>weight</code> , <code>style</code> , <code>display</code> , <code>stretch</code>	Promise of the family name
<code>injectCss(cssText, id)</code>	CSS text, optional id	The added <code><style></code> element
<code>cssUrl(spec) / css(spec)</code>	A spec	The stylesheet URL
<code>whenReady(family, weights)</code>	Family name, optional weights	Promise that resolves when the fonts are usable
<code>list()</code>	—	Promise of every family record
<code>search(query)</code>	Text	Promise of records whose name, designer or category matches
<code>get(idOrName)</code>	Id or name	Promise of the full metadata, or <code>null</code>
<code>slugify(name)</code>	Name	The family id
<code>base</code>	—	The base address (can be set)
<code>index</code>	—	The loaded catalogue, or <code>null</code> before <code>list()</code> , <code>search()</code> or <code>get()</code>

Spec forms

Form	Example
Name	<code>'Alegreya'</code>
Name with axis weights	<code>'Open Sans:wght@400;700'</code>
Name with weights and italic marker	<code>'Roboto:400,700i'</code>

Form	Example
Object	<code>{ family: 'Inter', weights: [400, 700] }</code>

Font Library snippets

Use & Embed tab

Snippet	Output
Link	<code><link rel="stylesheet" href=".../css/<id>.css"></code>
Import	<code>@import url('.../css/<id>.css');</code>
Script	The loader script tag with <code>data-families</code>
JS	<code>AuricFonts.load('<Family>')</code> with a <code>font-family</code> assignment
@font-face	Rules for up to four styles, with absolute file URLs
Usage	A <code>.headline</code> rule with the family, a fallback and a weight

Develop tab

Item	Output
Font-family stack (with safe fallbacks)	<code>font-family: '<Family>', <category fallbacks>;</code>
CSS variables	<code>:root { --font-<role>: <stack>; }</code> and a <code>body</code> rule
Tailwind	A <code>tailwind.config.js</code> <code>fontFamily</code> extension
SCSS	<code>\$font-<role>: <stack>;</code> and a <code>body</code> rule
Design tokens	JSON: <code>font.family.<role></code> with type <code>fontFamily</code>
CSS @import	The <code>@import</code> line and a <code>--font-<role></code> variable
Modular type scale	Base 10–24 px (16 default) and a ratio from 1.125 to 1.618; Copy CSS

`<role>` is `sans`, `serif` or `mono`, from the family's category. The fallback stacks are:

Category	Fallback stack
Sans-serif	<code>ui-sans-serif, system-ui, -apple-system, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif</code>
Serif	<code>ui-serif, Georgia, Cambria, 'Times New Roman', Times, serif</code>
Monospace	<code>ui-monospace, SFMono-Regular, Menlo, Consolas, 'Liberation Mono', monospace</code>
Display	<code>ui-sans-serif, system-ui, sans-serif</code>
Handwriting	<code>'Segoe Script', cursive</code>

The type-scale ratios are 1.125 (Major Second), 1.200 (Minor Third, the default), 1.250 (Major Third), 1.333 (Perfect Fourth), 1.414 (Augmented Fourth), 1.500 (Perfect Fifth) and 1.618 (Golden Ratio). The steps are named **xs**, **sm**, **base**, **md**, **lg**, **x1**, **2x1**, **3x1** and **4x1**.

Downloads

File	Contents
<code><Family>-font-kit.zip</code>	One family: <code>fonts/</code> , <code>css/<id>.css</code> , <code>demo.html</code> , <code>README.txt</code> , <code>LICENSE.txt</code>
<code>auric-font-selection-kit.zip</code>	The same for each family in your selection
<code>DOWNLOAD_NOTES.txt</code>	Added when a file could not be fetched

Licence permissions shown per family

The **License** tab marks each of these as allowed, not allowed or unknown: **Commercial use**, **Modification**, **Redistribution**, **Private / personal use**, **Web embedding** and **Sell font on its own**. It also shows the licence name, a **License URL** where known, the original licence file, the copyright, and the **Full license** text.

Related pages

- [Use the loader](#)
- [Licences](#)

SERVICES > FONT CDN

Troubleshooting

Why a Font CDN family might not appear or look right, and what to do.

Symptom	Why	What to do
The text stays in the fallback font	The stylesheet URL is wrong, so it returns a "not found" page	Open the stylesheet URL: it should start with a comment naming the family. Copy it from Use & Embed .
The text stays in the fallback font, and the URL is right	The CSS names the family differently	Use the exact name in the stylesheet's <code>font-family</code> , in quotes, for example <code>'Roboto Slab'</code>
The fallback shows briefly, then the font	Normal: stylesheets use <code>font-display: swap</code>	Preload or use <code>AuricFonts.load()</code> and wait for it before showing text that must not change
Bold looks smeared or too heavy	The family has no bold file, so the browser synthesises one	Pick a weight the family has; the Styles tab lists them
A weight has no effect on a static family	Only the weights the family has can be shown	Choose another weight or a variable family

Symptom	Why	What to do
Italic looks slanted rather than true italic	The family has no italic style	Check the Styles tab, or choose a family with italics
<code>data-families</code> loads nothing	A family name is misspelt, or the names are not separated by a vertical bar	Use the exact family names; check the browser's network panel for a failed <code>/css/</code> request
<code>AuricFonts</code> is undefined	Your code ran before the script loaded	Put your code after the script tag, or wait for the page's <code>load</code> event
<code>get()</code> returns <code>null</code>	No family has that id or name	Check the spelling, or use <code>search()</code>
<code>list()</code> or <code>search()</code> is slow the first time	The catalogue index is downloaded on first use	Call it once and reuse the result
Download kit reads Installed only	The family is a system font the library does not serve	Choose another family, or license the font from its maker
The kit has <code>DOWNLOAD_NOTES.txt</code>	One or more files could not be fetched	Try again later; the note lists what is missing
A licence shows as Unspecified	The library found no licence text for the family	Check with the foundry before using it; the License tab says "No bundled license text was found for this family"
Characters show as boxes or in another font	The family does not cover that script	Check the Charset tab and add a fallback font that covers it
A strict Content-Security-Policy blocks the font	Your policy does not allow the CDN	Allow the CDN in <code>style-src</code> and <code>font-src</code> (and <code>script-src</code> for the loader), or self-host

Related pages

- [Embed a family](#)
- [Self-host a family](#)
- [FAQ](#)

SERVICES > FONT CDN

FAQ

Short answers to common questions about the Auric Artisan Font CDN.

Is the Font CDN free?

The CDN's stylesheets, files and loader are served publicly, and the Font Library page needs no sign-in to copy embed code or download kits. Each font keeps its own licence terms; see [Licences](#).

Do I need an API key?

No.

How is this different from Google Fonts?

It works the same way, with a stylesheet per family, but everything is served from fonts.auricartisan.com, including fonts that came from other open sources, and every family comes with its licence text and a downloadable self-hosting kit.

Can I use these fonts in commercial projects?

Most families are under open licences that allow commercial use, such as the SIL Open Font License. Check each family's **License** tab: it shows the licence and a permissions summary.

Can I choose only some weights to download less?

The stylesheet always declares the whole family. Browsers generally download a font file only when the page uses that style. To ship exactly the files you want, self-host and remove the rules you do not need.

Can I use the fonts offline or behind a firewall?

Yes: download the kit and serve it from your own site. See [Self-host a family](#).

What does "Installed" mean?

The family is a system font listed for reference. Its licence does not allow redistribution, so the CDN does not serve it and it cannot be downloaded.

Are variable fonts supported?

Yes. 550 families are variable; their stylesheets declare a weight range, and width where the font has it.

Can I use a font file that is not in the library?

Yes, with `AuricFonts.face({ family, url })`, which registers any font file you have the rights to.

Where are the fonts' licences?

In each family's **License** tab, in the `LICENSE.txt` of each kit, in the stylesheet's header comment, and in the collection reference at <https://fonts.auricartisan.com/LICENSE.md>.

[SERVICES](#) > [FONT CDN](#)

Other material

Background material for the Font CDN — glossary, licences, limits and accuracy, related tools and privacy.

This folder holds the reference material around the Font CDN guides. The CDN has no keyboard shortcuts of its own and no public changelog; its catalogue build date is given in the [Reference](#).

Page	What it covers
Glossary	Type and web-font terms
Licences	The licences in the collection, what they allow, and how to comply
Limits and accuracy	What the catalogue's categories, scores and licence data can and cannot tell you
Related	The Font Library page, the VS Code Font Hub and other services
Privacy	What a page using the CDN requests, and what the loader does

[SERVICES](#) > [FONT CDN](#)

Licences

The licences that cover the fonts on the Font CDN, what they allow, where to find each family's licence, and how to comply.

Auric Artisan does not claim ownership of any font on the CDN. Each typeface remains the property of its authors and is redistributed under its original licence. The collection's licence reference, at <https://fonts.auricartisan.com/LICENSE.md>, lists the licences and reproduces their texts.

The licences in the collection

The reference describes the served collection as follows:

Licence	Commercial use	Redistribute	Modify	Main condition
SIL Open Font License 1.1 (the default)	Yes	Yes	Yes	Keep the licence; never sell the font alone; rename modified versions that use a Reserved Font Name
Apache License 2.0	Yes	Yes	Yes	Keep the licence and notices; state significant changes
CC0 1.0 / public domain	Yes	Yes	Yes	None
Bitstream Vera Fonts License	Yes	Yes	Yes	A modified version must be renamed without "Bitstream" or "Vera"

The catalogue's own statistics also count a few families under MIT and one under a custom licence, and some with no licence text found. The Font Library marks those **Unspecified** or **Custom**.

Where to find a family's licence

- **The License tab** in the Font Library shows the licence name, a **License URL** where known, the original licence file, the copyright, a permissions summary and the **Full license** text.
- **The stylesheet's header comment** names the licence and copyright, for example "SIL Open Font License 1.1 · Copyright 2011 The Alegreya Project Authors".
- **Download kits** include `LICENSE.txt` when the library has the text, and `README.txt` names the licence.
- **Use & Embed** ends with a note naming the family's licence.

The permissions summary

For each family, the License tab marks each of these as allowed, not allowed or unknown:

- Commercial use
- Modification
- Redistribution
- Private / personal use
- Web embedding
- Sell font on its own

The summary is a guide. The licence text is what counts.

Complying

1. Read the family's licence before you ship.
2. When you self-host, keep the licence file with the font files; the kit is organised to make this easy.
3. Do not sell OFL fonts on their own. Selling a product that uses them, such as a website, an app or a printed piece, is allowed.
4. If you modify an OFL font that has a Reserved Font Name, rename your version.
5. For **Unspecified** or **Custom** licences, check with the foundry first.
6. **Installed** families are not licensed for redistribution through the CDN; license them from their makers.

Note: This page summarises licences to help you find your way. It is not legal advice.

Related pages

- [Self-host a family](#)
- [Reference](#)

SERVICES > FONT CDN

Glossary

Definitions of the type and web-font terms used in the Font CDN documentation.

Axis. One dimension a variable font can vary along, such as weight (`wght`), width (`wdth`), slant (`slnt`) or optical size (`opsz`).

CDN (content delivery network). A service that serves files, here fonts and stylesheets, to web pages.

Fallback. A font the browser uses when the one you asked for is not available yet or lacks a character.

Family. A set of related fonts sharing a name, such as Alegreya in regular, italic and several weights.

Family id. The lower-case, hyphenated form of a family name used in its stylesheet URL, such as `roboto-slab`.

@font-face. The CSS rule that tells the browser where to find a font file and which family, weight and style it provides.

font-display: swap. A setting that shows text in a fallback font straight away and swaps to the web font when it arrives.

Font stack. A `font-family` list with fallbacks in order of preference.

Glyph. A single drawn character shape in a font.

Installed. A family listed in the catalogue but not served, because it is a system font whose licence does not allow redistribution.

Kit. A ZIP file from the Font Library with a family's font files, stylesheet, demo page, notes and licence, for self-hosting.

Loader. The `font-cdn.js` script that adds stylesheets and offers the `AuricFonts` JavaScript API.

OpenType features. Optional behaviours built into a font, such as ligatures, small capitals or tabular figures, switched on with CSS `font-feature-settings`.

SIL Open Font License (OFL). The most common open licence for fonts. It allows use, embedding, modification and redistribution, but not selling the font file on its own.

Self-hosting. Serving font files from your own site instead of a CDN.

Static font. A font file with one fixed weight and style.

Variable font. A font file that contains a continuous range along one or more axes, such as every weight from 100 to 900.

Weight. How heavy the strokes are, from 100 (Thin) to 900 (Black); 400 is Regular and 700 is Bold.

SERVICES > FONT CDN

Limits and accuracy

What the Font CDN's catalogue data — categories, scores, licences, script coverage — can and cannot tell you, and the CDN's practical limits.

Catalogue data

- **Metadata comes from the font files themselves:** names, designers, versions, weights, axes, character coverage, OpenType features and embedding flags are read from each font's internal tables. If a font's own data is wrong or missing, the catalogue is too.
 - **Categories are computed, not curated.** Serif or sans is worked out from the outlines of letters such as I, H and L when a font does not declare it. A few families, such as rounded sans-serifs, can land in the wrong category.
 - **Scores** (accessibility, UI, readability, developer and editorial) are calculated from measurable features such as x-height and how distinct confusable characters like l, I and 1 are. They are a starting point for choosing, not a verdict, and a few display fonts score higher than they should.
 - **Licences** are detected from the licence files bundled with each family and from the font's name data. A family with no licence text found is marked **Unspecified**; check it yourself.
 - **Script coverage** lists the Unicode blocks a font has characters in. Having characters in a block does not guarantee complete or correct support for a language.
-

Serving

- Stylesheets declare the whole family; there is no per-weight or per-character subsetting in the URL.
 - Files are served in the formats the fonts were distributed in, such as TrueType and OpenType; the **Styles** tab lists each file with its size.
 - System fonts marked **Installed** are listed but not served.
 - The catalogue changes only when it is rebuilt; the current build is from 2 June 2026.
-

The loader

- `list()`, `search()` and `get()` download catalogue data (the index is large) on first use.
- `load()` waits only for the weights you name, or for weight 400 if you name none.
- `face()` does not report a failed download as an error; check that the font appears.

Related pages

- [Licences](#)
- [Reference](#)

SERVICES > FONT CDN

Privacy

What a page that uses the Font CDN requests from it, and what the loader script does and does not do.

What your visitors' browsers request

A page that uses the CDN makes requests to <https://fonts.auricartisan.com> and nowhere else for its fonts:

You use	Your visitors' browsers request
A link tag or <code>@import</code>	The family stylesheet, then the font files the page actually uses
The loader with <code>data-families</code> or <code>load()</code>	The loader script, the family stylesheets, then the font files
<code>list()</code> , <code>search()</code> or <code>get()</code>	Also the catalogue data files

As with any file host, the CDN receives the ordinary information that comes with each request.

What the loader does not do

The loader served at <https://fonts.auricartisan.com/font-cdn.js>:

- sets no cookies;
- writes nothing to the browser's storage;
- makes no requests other than the stylesheets and data files described above.

Self-hosting

If you prefer your visitors' browsers never to contact the CDN, download a kit and serve the files from your own site. See [Self-host a family](#).

The Font Library page

Previews on the Font Library page are drawn in your browser from the CDN's files, and download kits are assembled in your browser. For the auricartisan.com website as a whole, see the site's privacy policy at <https://auricartisan.com/legal/privacy/>.

SERVICES > FONT CDN

Related

The Font Library page, the VS Code Font Hub, the Icon CDN and other related Auric Artisan products.

Product	Why	Docs
Font Library (website)	Browse, preview, compare and pair families; copy embed code; download kits	Font Library
Auric Artisan Font Hub (VS Code)	Work with fonts from inside VS Code	Font Hub
Icon CDN	Icons served the same way, from icons.auricartisan.com	Icon CDN
Design Engine (website)	Build a design system with fonts and colours	Design Engine
Public API	Colour and accessibility checks for your code	Public API

Links:

- Font Library: <https://auricartisan.com/tool/font/>
- Licence reference: <https://fonts.auricartisan.com/LICENSE.md>

SERVICES > ICON CDN

Icon CDN

What the Auric Artisan Icon CDN at icons.auricartisan.com serves, who it is for, and how to put an icon on your page in a few steps.

The Auric Artisan Icon CDN, at <https://icons.auricartisan.com>, serves 12,896 icons from seven open-source sets, in 32,803 variants: Lucide, Phosphor, Tabler, Remix, Heroicons, Feather and Material. A small loader script turns placeholders such as `<i data-icon="lucide:house">` into inline SVG you can colour and size with CSS, and a JavaScript API returns any icon's markup or searches the catalogue. The SVG files are also served directly, for use in `<img>` tags or your own build.

Every set keeps its own open-source licence (MIT, ISC, Apache 2.0 or the Remix Icon License), all of which allow commercial use. The icons are drawn with `currentColor`, so they take the colour of the surrounding text unless you set one.

You browse, recolour and resize icons, copy their code, and export selections as a ZIP or an SVG sprite in the Icon Library at <https://auricartisan.com/tool/icon/>. That page is documented with the website tools; these pages cover the CDN itself: the collections, the loader, embedding and licences.

Who it is for

- **Web developers** who want icons in plain HTML without a build step or an npm package.
- **Designers and prototypers** who want to try icons from several sets under one name scheme.
- **React developers** who want a quick JSX component for a single icon.
- **Teams that self-host** and want to download SVGs or sprites.

Where to find it

What	Where
CDN host	https://icons.auricartisan.com
Loader script	https://icons.auricartisan.com/icon-cdn.js
SVG files	<a href="https://icons.auricartisan.com/icon/<set>/...">https://icons.auricartisan.com/icon/<set>/...
Browse and copy code	https://auricartisan.com/tool/icon/
Licence reference	https://icons.auricartisan.com/LICENSE.md

Quick start

1. Add the loader to your page:

```
<script src="https://icons.auricartisan.com/icon-cdn.js"></script>
```

2. Place an icon where you want it:

```
<i data-icon="lucide:house"></i>
```

3. Load the page. The placeholder is replaced by the inline SVG.

4. Size and colour it with attributes, or with CSS on the element:

```
<i data-icon="phosphor:heart" data-style="fill" data-size="32" data-color="#e11"></i>
```

5. To find icon names, search the Icon Library at <https://auricartisan.com/tool/icon/>, open an icon and use **Copy CDN tag**.

6. Keep the set's licence notice where its licence asks for it; see [Licences](#).

What you can do

- Turn `<i data-icon="set:name">` placeholders into inline, themeable SVG.
- Choose a style (such as Phosphor's thin to duotone weights, or Tabler's outline and filled), a size, a colour and a stroke width.
- Get an icon's markup from JavaScript with `AuricIcons.svg()`, or search the catalogue with `AuricIcons.search()`.
- Link SVG files directly, or use the Material and Remix webfonts.
- Copy SVG, JSX, a data-URI `<img>` or a CDN tag from the Icon Library, and export selections as a ZIP or a `<symbol>` sprite.

In this folder

- [docs/](#)
 - [Getting started](#)
 - [Collections](#)
 - [Embed an icon](#)
 - [Use the loader](#)
 - [Style, size and colour](#)
 - [Export and self-host](#)
 - [Reference](#)
 - [Troubleshooting](#)
 - [FAQ](#)
- [others/](#)

- [Glossary](#)
- [Licences](#)
- [Limits and accuracy](#)
- [Related](#)
- [Privacy](#)

SERVICES > ICON CDN

Icon CDN documentation

The contents of the Icon CDN documentation, from a first icon to the loader API, styling and self-hosting.

These pages explain how to use icons from icons.auricartisan.com on your own pages: the icon sets and their styles, the loader and its JavaScript API, other ways to embed an icon, styling, and exporting icons to host yourself.

Page	What it covers
Getting started	How the CDN is organised, icon names, and a first icon
Collections	The seven sets: styles, counts, grids and licences
Embed an icon	Loader placeholders, inline SVG, JSX, data-URI images, direct files and webfonts
Use the loader	The <code>icon-cdn.js</code> script and the <code>AuricIcons</code> API
Style, size and colour	Choosing a style, sizing, colouring and stroke width
Export and self-host	ZIP and sprite exports, single SVG downloads, hosting the loader yourself
Reference	Every URL pattern, attribute, method and option
Troubleshooting	Icons that do not appear or look wrong
FAQ	Short answers

SERVICES > ICON CDN

Getting started

How the Icon CDN is organised, how icons are named, and how to put a first icon on a page.

How the CDN is organised

Everything is served from <https://icons.auricartisan.com>:

Path	What it is
<code>/icon-cdn.js</code>	The loader script
<code>/icon/<set>/...</code>	The original SVG files, and the Material and Remix webfonts
<code>/data/sets.json</code>	The seven sets: styles, licence, grid and counts
<code>/data/index.json</code>	One record per icon, used for names and search
<code>/data/bodies/<set>.<style>.json</code>	The icon markup for one set and style, loaded by the loader as needed
<code>/data/material.json</code>	The code points for the Material webfont
<code>/LICENSE.md</code>	The licence reference for every set

Icon names

An icon is named by its set and its name, joined by a colon: `lucide:house`, `tabler:home`, `phosphor:heart`. The loader also accepts:

Form	Example	Meaning
<code>set:name</code>	<code>tabler:home</code>	This icon from this set
<code>set-name</code>	<code>tabler-home</code>	The same
<code>name</code>	<code>home</code>	The first set with this name: Lucide, Phosphor, Tabler, Remix, Heroicons, Feather, then Material

Names are the set's own icon names in lower case with hyphens. Different sets name things differently: Lucide calls it `house`, Tabler calls it `home`. The Icon Library shows every icon's name and gives you the exact tag with **Copy CDN tag**.

Your first icon

1. Add the loader once, anywhere in the page:

```
<script src="https://icons.auricartisan.com/icon-cdn.js"></script>
```

2. Add a placeholder where the icon should go:

```
<button><i data-icon="lucide:house"></i> Home</button>
```

3. Load the page. When the page has loaded, the loader replaces the contents of every `data-icon` element with the icon's SVG.

The icon uses `currentColor`, so it takes the button's text colour.

Important: The SVG markup the CDN serves has a `viewBox` but no width or height of its own. Without a size, browsers draw it at their default size for SVG, which is usually far too large. Always give icons a size, either with `data-size` or with CSS such as the rule below.

```
[data-icon] svg { width: 1em; height: 1em; }
```

See [Style](#), [size](#) and [colour](#).

Next steps

- [Collections](#), to choose a set and a style.

- **Embed an icon**, for ways that do not use the loader.
- **Use the loader**, for icons added after the page loads.

SERVICES > ICON CDN

Collections

The seven icon sets on the Icon CDN — their styles, icon counts, grids, whether they are stroke-based, and their licences.

The CDN serves seven sets: 12,896 icons in 32,803 variants. An **icon** is one symbol, such as "heart"; a **variant** is one style of it, such as Phosphor's heart in bold or fill.

Set	Icons	Variants	Styles	Default style	Licence
Lucide	1,714	1,714	regular	regular	ISC
Phosphor	1,512	9,072	thin, light, regular, bold, fill, duotone	regular	MIT
Tabler	5,093	6,146	outline, filled	outline	MIT
Remix	1,690	3,229	line, fill	line	Remix Icon License v1.0
Heroicons	324	1,288	24/outline, 24/solid, 20/solid, 16/solid	24/outline	MIT
Feather	287	287	regular	regular	MIT
Material	2,276	11,067	outlined, round, sharp, regular, twotone	outlined	Apache 2.0

Grids and strokes

Set	Grid (viewBox)	Drawn with
Lucide	24 × 24	Strokes
Phosphor	256 × 256	Fills
Tabler	24 × 24	Strokes
Remix	24 × 24	Fills
Heroicons	24, 20 or 16, by style	Strokes (outline) and fills (solid)
Feather	24 × 24	Strokes
Material	Webfont	Glyphs of a font

Stroke width (**data-stroke**) changes only icons drawn with strokes: Lucide, Tabler, Feather and Heroicons' outline style.

Material is a webfont

Material icons are not SVG: the CDN serves them as five webfonts (Regular, Outlined, Round, Sharp and Two-Tone) with their code points. The loader renders a Material icon as a character in the matching font, so it behaves like text: its size follows `data-size` or the font size, and its colour follows `data-color` or the text colour.

Remix categories

Remix icons are organised in twenty folders on the CDN, such as `Arrows`, `Buildings`, `Business`, `Communication`, `Design`, `Development`, `Device`, `Document`, `Editor`, `Finance`, `Food`, `Game & Sports`, `Health & Medical`, `Logos`, `Map`, `Media`, `Others`, `System`, `User & Faces` and `Weather`. Remix also ships its own webfont; see [Embed an icon](#).

Categories in the Icon Library

The Icon Library groups icons into categories such as Arrows, Brand, Design, Devices, Document, E-commerce, Map, Media, System and Weather, where the sets provide them. Many icons have no category.

Upstream sources

Set	Home	Credit
Lucide	https://lucide.dev	Lucide Contributors
Phosphor	https://phosphoricons.com	Phosphor Icons
Tabler	https://tabler.io/icons	Paweł Kuna
Remix	https://remixicon.com	Remix Design
Heroicons	https://heroicons.com	Tailwind Labs
Feather	https://feathericons.com	Cole Bemis
Material	https://fonts.google.com/icons	Google

Related pages

- [Licences](#)
- [Style, size and colour](#)

SERVICES > ICON CDN

Embed an icon

Every way to put an Icon CDN icon on a page — loader placeholders, inline SVG, a React component, a data-URI image, direct SVG files and the Material and Remix webfonts.

The Icon Library at <https://auricartisan.com/tool/icon/> gives every icon ready-made code. Open an icon, set its style, size and colour, then choose a copy button or copy a block from the code panel. This page explains each option.

CDN tag (the loader)

Copy CDN tag gives a placeholder and the loader script:

```
<i data-icon="phosphor:heart" data-style="fill"></i>
<script src="https://icons.auricartisan.com/icon-cdn.js"></script>
```

data-style appears only when you chose a style other than the set's default. Include the script once per page, and use as many placeholders as you like. The loader replaces each placeholder's contents with inline SVG when the page loads; see [Use the loader](#).

Best for: plain HTML pages, prototypes, content managed in a CMS.

Inline SVG

Copy SVG gives the icon's SVG markup, with your chosen colour and stroke applied. Paste it straight into your HTML. Nothing needs to load from the CDN afterwards.

```
<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><polyline points="22 12 18 12 15 21 9 3 6 12 2 12"/></svg>
```

Give it a size with **width** and **height** attributes or CSS.

Best for: a handful of icons you want in your own markup, with no script.

JSX / React

Copy JSX gives a React component. Attribute names are converted to React's camelCase, and your props are spread onto the **<svg>**:

```
export const Activity = (props) => (
  <svg {...props} viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
  strokeLinejoin="round"><polyline points="22 12 18 12 15 21 9 3 6 12 2 12"/></svg>
);
```

Use it as **<Activity width={24} height={24} />**.

Image with a data URI

Copy data-URI gives an `<img>` whose source is the SVG itself, encoded into the address, with your chosen size and the icon's name as its `alt` text:

```

```

An image cannot inherit the page's text colour, so choose the colour before you copy. Change the `alt` text to describe what the icon means in your page, or use `alt=""` if the icon is decorative.

Direct SVG files

Every SVG icon is also served as a file you can link:

```

```

Paths follow each set's own folders:

Set	Example path
Feather	<code>/icon/feather/activity.svg</code>
Lucide	<code>/icon/lucide/house.svg</code>
Tabler	<code>/icon/tabler/outline/home.svg</code> , <code>/icon/tabler/filled/home.svg</code>
Phosphor	<code>/icon/phosphor/regular/heart.svg</code> , <code>/icon/phosphor/fill/heart-fill.svg</code>
Heroicons	<code>/icon/heroicons/24/outline/home.svg</code>
Remix	<code>/icon/remix/Buildings/home-line.svg</code>

As with a data URI, a linked file does not take the page's text colour.

Download .svg in the Icon Library saves the icon, in your chosen colour and stroke, as `<id>-<style>.svg`.

Webfonts

Material

Material icons are glyphs in a webfont. The Icon Library's code panel for a Material icon offers **HTML (CDN)**, which uses the loader:

```
<i data-icon="material:home" data-style="outlined"></i>  
<script src="https://icons.auricartisan.com/icon-cdn.js"></script>
```

and **Webfont @font-face**, which declares the font from the CDN and shows the icon's character code, so you can use the font directly. **Copy glyph** copies the character entity.

Remix

Remix Icon's own webfont stylesheet is served at

<https://icons.auricartisan.com/icon/remix/fonts/remixicon.css>. It uses Remix's **ri-*** classes, as documented by the Remix Icon project.

Accessibility

- An icon next to visible text is usually decorative: hide it from assistive technology, for example with **aria-hidden="true"** on the placeholder.
- An icon on its own, such as an icon-only button, needs an accessible name, for example **aria-label** on the button.

Related pages

- [Use the loader](#)
- [Style, size and colour](#)
- [Export and self-host](#)

SERVICES > ICON CDN

Export and self-host

Export icons from the Icon Library as a ZIP of SVG files or an SVG sprite, download single icons, and host icons or the loader yourself.

You do not have to load icons from icons.auricartisan.com at run time. The Icon Library can export them for your own project.

Collect icons in your kit

1. In the Icon Library at <https://auricartisan.com/tool/icon/>, open an icon and choose **+ Select**.
2. Selected icons collect in the **Your kit** tray, with a count. The tray notes that it is "Kept in this browser".
3. Choose **Hide** to put the tray away, and **Clear** to empty it.

Download a ZIP

Choose **Download ZIP** in the tray. The file **auric-icons.zip** contains:

- one SVG per icon, in a folder per set, named **<name>-<style>.svg**;

- a `README.txt` noting the number of icons and that each uses `currentColor`;
- for Material icons, a text file per icon with its webfont character code instead of an SVG.

Copy an SVG sprite

Choose **Copy sprite** in the tray. The Icon Library copies one hidden `<svg>` containing a `<symbol>` for each selected icon, with the icon's id as its `id`:

```
<svg xmlns="http://www.w3.org/2000/svg" style="display:none">
  <symbol id="lucide-house" viewBox="0 0 24 24">...</symbol>
</svg>
<!-- use: <svg><use href="#ID"/></svg> -->
```

Paste it once near the top of your page, then use any icon with:

```
<svg width="24" height="24"><use href="#lucide-house"/></svg>
```

A sprite needs no script and no further requests, and icons in it still take `currentColor`.

Download one icon

Open an icon and choose **Download .svg**. The file is named `<id>-<style>.svg` and uses your chosen colour and stroke.

Link files directly

Every SVG icon is also served as a file under `https://icons.auricartisan.com/icon/<set>/...`; see [Embed an icon](#). You can download files from these addresses to host them yourself.

Host the loader yourself

The loader works from any address. To serve it from your own site:

1. Copy `icon-cdn.js` and the `data/` files it uses (`sets.json`, `index.json`, the `bodies/` files for the sets and styles you need, and `material.json` with the Material fonts if you use Material) to a folder on your site, keeping the same layout.
2. Load the script from there. It finds its data next to itself, or set `data-base` on the script tag to point elsewhere.

See [Use the loader](#).

Keep the licences

When you ship icons, keep each set's licence notice where its licence asks for it. See [Licences](#).

Related pages

- [Embed an icon](#)
- [Licences](#)

SERVICES > ICON CDN

Style, size and colour

Choose an icon's style, give it a size, colour it and change its stroke width, with the loader, CSS or the Icon Library.

Style

Many icons come in several styles: Phosphor's **thin**, **light**, **regular**, **bold**, **fill** and **duotone**; Tabler's **outline** and **filled**; Remix's **line** and **fill**; Heroicons' **24/outline**, **24/solid**, **20/solid** and **16/solid**; Material's **outlined**, **round**, **sharp**, **regular** and **twotone**. See [Collections](#).

- With the loader, set **data-style="fill"** or pass **{ style: 'fill' }** to **svg()**.
 - If you ask for a style the icon does not have, the loader uses the set's default style, or else the icon's first style.
 - In the Icon Library, the **Style** buttons in an icon's panel switch between its styles, and the copied code follows.
-

Size

Served icons have no width or height of their own, only a **viewBox**, so always size them.

With the loader:

```
<i data-icon="lucide:house" data-size="24"></i>      <!-- 24 px -->
<i data-icon="lucide:house" data-size="1.25em"></i> <!-- scales with text -->
```

With CSS, for every icon at once:

```
[data-icon] svg { width: 1em; height: 1em; vertical-align: -0.125em; }
```

Sizing in **em** makes icons follow the surrounding font size. Material icons are font glyphs and follow **data-size** or the font size.

Icons are drawn on a grid: 24 units for most sets, 256 for Phosphor, and 20 or 16 for some Heroicons styles.

In the Icon Library, the **Size** slider runs from 16 to 160 px and sets the size of the copied `<img>` code.

Colour

Icons are drawn with **currentColor**: by default they take the text colour of the element they are in.

- To match your text, do nothing, or set **color** on the parent in CSS.
- To set a colour with the loader, use `data-color="#e11"` or `{ color: '#e11' }`.
- With CSS: `.danger [data-icon] { color: #b00020; }`.

A linked SVG file or a data-URI image cannot inherit the page's colour. Choose the colour in the Icon Library before you copy, or recolour the file yourself. In the Icon Library, **Colour** sets a colour and the **currentColor** button goes back to inheriting.

Stroke width

Stroke-drawn icons (Lucide, Tabler, Feather, and Heroicons' outline style) can be made thinner or bolder:

- with the loader, `data-stroke="1.5"` or `{ stroke: 1.5 }`;
- with CSS, `[data-icon] svg { stroke-width: 1.5; }`.

In the Icon Library, the **Stroke** slider runs from 0.5 to 3 in steps of 0.25, or **auto** for the set's own width. Fill-drawn sets, such as Phosphor and Remix, ignore stroke width; choose a lighter or heavier style instead.

Related pages

- [Collections](#)
- [Use the loader](#)

SERVICES > ICON CDN

Use the loader

Render Icon CDN icons with the icon-cdn.js placeholders and the AuricIcons JavaScript API, including icons added after the page loads.

icon-cdn.js is a small script with no dependencies. It fills **data-icon** placeholders with inline SVG, returns any icon's markup to your code, and searches the catalogue. It creates a global object, **AuricIcons** (also available as **AuricIconLoader**).

Placeholders

```
<script src="https://icons.auricartisan.com/icon-cdn.js"></script>

<i data-icon="lucide:house"></i>
<i data-icon="phosphor:heart" data-style="fill" data-size="32"
  data-color="#e11" data-stroke="1.5"></i>
```

Attribute	What it does	Example
<code>data-icon</code>	The icon to show (required): <code>set:name</code> , <code>set-name</code> or <code>name</code>	<code>tabler:home</code>
<code>data-style</code>	One of the set's styles; otherwise the set's default	<code>fill</code> , <code>outline</code> , <code>24/solid</code>
<code>data-size</code>	Width and height. A number is treated as pixels; a value with a unit is used as written.	<code>24</code> , <code>1.5em</code>
<code>data-color</code>	The icon's colour; otherwise the text colour	<code>#e11</code> , <code>rebeccapurple</code>
<code>data-stroke</code>	Stroke width, for stroke-drawn icons	<code>1.5</code>

When the page has finished loading (or at once, if it already has), the loader replaces the contents of every element with `data-icon` and marks it with a `data-icon-done` attribute. An element marked done is not processed again. If the icon cannot be found, the element is left as it was.

Any element works, not only `<i>`: `<span data-icon="...">` is fine.

Icons added later

Placeholders added after the page loads, for example by your framework, are not filled automatically. Call `inject()` after adding them:

```
await AuricIcons.inject();           // the whole document
await AuricIcons.inject(myContainer); // only inside one element
```

It returns the number of placeholders it found.

Get an icon's markup

```
const svg = await AuricIcons.svg('tabler:home', { size: 24, color: 'red' });
element.innerHTML = svg;
```

Option	Meaning
<code>style</code>	One of the set's styles

Option	Meaning
<code>size</code>	Width and height, as for <code>data-size</code>
<code>color</code>	The colour
<code>stroke</code>	The stroke width

`svg()` returns a string: SVG markup for SVG sets, or a `<span>` in the Material webfont for Material icons. It returns an empty string when there is no such icon.

Find icons

```
const hits = await AuricIcons.search('arrow', { set: 'lucide', limit: 50 });
const rec  = await AuricIcons.get('feather:activity');
const sets = await AuricIcons.sets();
```

Method	Returns
<code>search(query, { set, limit })</code>	Records whose name, id or search terms contain the query, optionally in one set and limited in number
<code>get(name)</code> or <code>resolve(name)</code>	The icon's record (its set, id, name and available styles), or <code>null</code>
<code>list()</code>	Every icon record
<code>sets()</code>	The metadata of the seven sets

The first call to any of these downloads the catalogue index, which is large; reuse the results.

How names are resolved

1. With a set prefix (`tabler:home` or `tabler-home`), the loader looks for that icon in that set.
2. If that set has no such icon, or there is no prefix, it takes the first set that has an icon with that name, in the order Lucide, Phosphor, Tabler, Remix, Heroicons, Feather, Material.
3. Finally it tries an exact id or name.

Because of step 2, a name that a set does not have can quietly come from another set. For example, Lucide's house icon is `house`, so `lucide:home` shows another set's `home` icon. Copy names from the Icon Library to be sure.

Where it loads from

By default the loader loads its data from the address it was loaded from. To use a self-hosted copy, set one of these (the first found wins):

1. `data-base` on the script tag;

2. `window.AURIC_ICON_BASE` (or `window.AURIC_ICON_ASSET_BASE`), set before the script loads;
3. `<meta name="auric-icon-base" content="...">` in the page.

You can also set `AuricIcons.base` from JavaScript.

Related pages

- [Style, size and colour](#)
- [Reference](#)
- [Privacy](#)

SERVICES > ICON CDN

Reference

Every URL pattern, set, style, loader attribute, method and export of the Auric Artisan Icon CDN.

Catalogue

Fact	Value
Host	https://icons.auricartisan.com
Sets	7
Icons	12,896
Variants	32,803
Catalogue built	3 June 2026
Icons by licence	MIT 7,216; ISC 1,714; Remix v1.0 1,690; Apache 2.0 2,276

URL patterns

Pattern	What it serves
<code>/icon-cdn.js</code>	The loader
<code>/icon/<set>/...</code>	SVG files, and the Material and Remix webfonts
<code>/icon/remix/fonts/remixicon.css</code>	Remix Icon's own webfont stylesheet
<code>/data/sets.json</code>	Set metadata: label, licence, styles, default style, grid, counts, home, credit
<code>/data/index.json</code>	One record per icon: id, name, set, styles, category, search terms
<code>/data/bodies/<set>.<style>.json</code>	SVG markup for one set and style (a <code>/</code> in a style becomes <code>-</code>)
<code>/data/material.json</code>	Material code points by style

Pattern	What it serves
<code>/data/stats.json</code> , <code>/data/meta.json</code>	Catalogue statistics and build information
<code>/LICENSE.md</code>	The licence reference

Sets and styles

Set	Styles (default first)	Grid	Stroke width applies
lucide	regular	24	Yes
phosphor	regular, thin, light, bold, fill, duotone	256	No
tabler	outline, filled	24	Yes
remix	line, fill	24	No
heroicons	24/outline, 24/solid, 20/solid, 16/solid	24, 20, 16	Outline only
feather	regular	24	Yes
material	outlined, round, sharp, regular, twotone	Webfont	No

Name resolution order when no set is given: Lucide, Phosphor, Tabler, Remix, Heroicons, Feather, Material.

Placeholder attributes

Attribute	Values	Default
<code>data-icon</code>	<code>set:name</code> , <code>set-name</code> or <code>name</code>	Required
<code>data-style</code>	A style of the set	The set's default
<code>data-size</code>	A number (pixels) or a CSS length	None: size it with CSS
<code>data-color</code>	Any CSS colour	<code>currentColor</code>
<code>data-stroke</code>	A number	The set's own width

Filled placeholders get a `data-icon-done` attribute and are not processed again.

Script configuration

Setting	Meaning
<code>data-base</code> on the script	The address to load data from
<code>window.AURIC_ICON_BASE</code> / <code>window.AURIC_ICON_ASSET_BASE</code>	The same, set before the script loads
<code><meta name="auric-icon-base"></code>	The same, in the page

AuricIcons API

The global `AuricIcons` (alias `AuricIconLoader`).

Member	Parameters	Returns
<code>svg(name, options)</code>	<code>style</code> , <code>size</code> , <code>color</code> , <code>stroke</code>	Promise of the markup string, or <code>''</code> if not found
<code>inject(root)</code>	An element; default the document	Promise of the number of placeholders found
<code>get(name)</code> / <code>resolve(name)</code>	A name	Promise of the icon record, or <code>null</code>
<code>search(query, options)</code>	Text; <code>set</code> , <code>limit</code>	Promise of matching records
<code>list()</code>	—	Promise of every record
<code>sets()</code>	—	Promise of the set metadata
<code>slug(text)</code>	Text	Lower-case, hyphenated text
<code>base</code>	—	The data address (can be set)
<code>index</code>	—	The loaded index, or <code>null</code> before first use

Icon Library outputs

Control	Output
Copy SVG	The SVG markup, with the chosen colour and stroke
Copy JSX	A React component that spreads its props onto the <code><svg></code>
Copy data-URI	An <code></code> with the SVG as a data URI, the chosen size and the icon name as <code>alt</code>
Copy CDN tag	A placeholder with <code>data-icon</code> (and <code>data-style</code> if not the default) and the loader script
Download .svg	<code><id>-<style>.svg</code>
Copy glyph (Material)	The character entity for the webfont
Code panel (SVG sets)	SVG, JSX / React, <code></code> data-URI, CDN (auto-inject)
Code panel (Material)	HTML (CDN), Webfont @font-face
Download ZIP (tray)	<code>auric-icons.zip</code> : SVGs by set, a README, and text files for Material icons
Copy sprite (tray)	A hidden <code><svg></code> with one <code><symbol></code> per selected icon

Icon Library controls

Control	Range
Size	16–160 px

Control	Range
Stroke	0.5–3 in steps of 0.25, or auto
Colour	Any colour, or currentColor
Style	The icon's styles

Related pages

- [Use the loader](#)
- [Collections](#)

SERVICES > ICON CDN

Troubleshooting

Why an Icon CDN icon might not appear, appear huge, show the wrong icon or ignore its colour, and what to do.

Symptom	Why	What to do
Icons are huge, or fill their container	Served SVGs have no width or height	Set <code>data-size</code> , or add CSS such as <code>[data-icon] svg { width: 1em; height: 1em; }</code>
A placeholder stays empty	The name matches no icon	Copy the name from the Icon Library; check the spelling and the set
The wrong set's icon appears	The set you named has no icon by that name, so another set's is used	Use the set's own name, for example <code>lucide:house</code> rather than <code>lucide:home</code>
Icons added after page load stay empty	The loader fills placeholders once, when the page loads	Call <code>AuricIcons.inject()</code> after adding them
An icon does not update when I change <code>data-icon</code>	Filled placeholders are marked <code>data-icon-done</code>	Remove that attribute (and the contents), then call <code>inject()</code> again
The colour does not change	An <code></code> or linked SVG file cannot inherit text colour	Use the loader, inline SVG or a sprite; or pick the colour before copying
<code>data-stroke</code> has no effect	The set is drawn with fills	Choose a lighter or heavier style instead, for example Phosphor <code>light</code> or <code>bold</code>
A style is ignored	The icon does not have that style	Check the icon's Style buttons; the loader falls back to the default
A Material icon looks like a letter or box	The Material font has not loaded yet, or the name is wrong	Wait for the font; check the name in the Icon Library
The first icons are slow to appear	The loader downloads the catalogue index and the markup for each set and style on first use	Use fewer sets and styles, or export a sprite

Symptom	Why	What to do
<code>AuricIcons</code> is undefined	Your code ran before the script loaded	Put your code after the script tag, or wait for the page's <code>load</code> event
A strict Content-Security-Policy blocks it	Your policy does not allow the CDN	Allow <code>https://icons.auricartisan.com</code> for <code>script-src</code> and <code>connect-src</code> (and <code>font-src</code> for Material), or self-host

Related pages

- [Use the loader](#)
- [Style, size and colour](#)
- [FAQ](#)

SERVICES > ICON CDN

FAQ

Short answers to common questions about the Auric Artisan Icon CDN.

Do I need an account or a key?

No. The loader, data and files are served publicly, and the Icon Library needs no sign-in.

Can I use the icons commercially?

Yes. All seven sets use licences that allow commercial use, redistribution and modification: MIT, ISC, Apache 2.0 and the Remix Icon License. Keep the licence notices as each licence asks. See [Licences](#).

Who made the icons?

Their original authors: Lucide Contributors, Phosphor Icons, Paweł Kuna (Tabler), Remix Design, Tailwind Labs (Heroicons), Cole Bemis (Feather) and Google (Material). Auric Artisan does not claim ownership of any icon.

Why use this instead of each set's own package?

One loader and one naming scheme cover all seven sets, with no build step. If you use a single set heavily in a built app, that set's own package may suit you better.

Does the loader work with React, Vue or other frameworks?

The loader fills placeholders in the page. With a framework that renders after page load, call `AuricIcons.inject()` after rendering, or use **Copy JSX** for React components, or a sprite.

How do I make icons follow my text colour?

Do nothing: icons use `currentColor`. Avoid `data-color` and linked image files, which fix the colour.

Can I use icons offline?

Yes: export a ZIP or a sprite from the Icon Library, or host the loader and its data yourself. See [Export and self-host](#).

Why is my icon so big?

Served SVGs have no size of their own. Set `data-size` or a CSS width and height.

Is Material Icons SVG?

No. The CDN serves Material as webfonts, and the loader renders Material icons as font characters.

SERVICES > ICON CDN

Other material

Background material for the Icon CDN — glossary, licences, limits and accuracy, related tools and privacy.

This folder holds the reference material around the Icon CDN guides. The CDN has no keyboard shortcuts of its own and no public changelog; its catalogue build date is given in the [Reference](#).

Page	What it covers
Glossary	Icon and SVG terms
Licences	Each set's licence, what it allows, and how to comply
Limits and accuracy	What the catalogue's names, categories and counts can and cannot tell you
Related	The Icon Library page, VS Code icon extensions and other services
Privacy	What a page using the CDN requests, and what the loader does

SERVICES > ICON CDN

Licences

The licence of each icon set on the Icon CDN, what it allows, and how to comply when you use or ship the icons.

Every icon on the CDN is free for commercial use, redistribution, embedding and modification, under the conditions of its set's own open-source licence. Auric Artisan does not claim ownership of any icon: each set remains the property of its authors and is redistributed under its original licence. The licence reference at <https://icons.auricartisan.com/LICENSE.md> gives the details and the full licence texts.

Licences by set

Set	Licence	Copyright
Phosphor	MIT	Phosphor Icons
Feather	MIT	Cole Bemis
Heroicons	MIT	Tailwind Labs, Inc.
Tabler	MIT	Paweł Kuna
Lucide	ISC	Lucide Icons and Contributors
Material	Apache License 2.0	Google LLC
Remix	Remix Icon License v1.0	Remix Design

What each licence asks

Licence	Commercial use	Redistribute	Modify	Main condition
MIT	Yes	Yes	Yes	Keep the copyright and licence notice
ISC	Yes	Yes	Yes	Keep the copyright and licence notice
Apache 2.0	Yes	Yes	Yes	Keep the licence and notices; state significant changes to the files
Remix Icon License v1.0	Yes	Yes	Yes	A permissive, Apache-2.0-style licence; see its text

Lucide is a community fork of Feather; the original Feather icons it derives from remain under MIT.

Brand icons

Some sets include logos of brands and companies. Those logos remain trademarks of their owners, and are provided for identification, not endorsement. Using a logo may need its owner's permission, whatever the set's licence says.

Complying

1. When you ship icons in a product or a download, include the relevant licence notices, for example in your credits or a licence file.
2. If you change Material icon files, note your changes, as Apache 2.0 requires.
3. Check brand and company logos separately.

Note: This page summarises licences to help you find your way. It is not legal advice.

Related pages

- [Collections](#)
- [Export and self-host](#)

SERVICES > ICON CDN

Glossary

Definitions of the icon and SVG terms used in the Icon CDN documentation.

currentColor. A CSS keyword meaning "the current text colour". Icons drawn with it take the colour of the text around them.

Data URI. A way to put a small file, here an SVG, directly into an address, so an `<img>` needs no separate download.

Default style. The style the loader uses when you do not choose one, such as **regular** for Phosphor.

Fill-drawn icon. An icon made of filled shapes. Its weight comes from its style, not from a stroke width.

Grid (viewBox). The coordinate square an icon is drawn in, such as 24×24 . It sets the icon's proportions, not its size on screen.

Icon. One symbol, such as "heart". **Variant**: one style of it, such as a bold or filled heart.

Inline SVG. SVG markup placed directly in the HTML, so CSS can colour and size it.

JSX. The HTML-like syntax React components are written in.

Loader. The `icon-cdn.js` script that fills `data-icon` placeholders and offers the **AuricIcons** API.

Placeholder. An element with a `data-icon` attribute that the loader fills with an icon.

Sprite. One hidden SVG containing many icons as `<symbol>` elements, each shown elsewhere with `<use href="#id">`.

Stroke-drawn icon. An icon made of lines, whose thickness can be changed with a stroke width.

Style. A version of a set's icons, such as outline or filled, or a weight such as thin or bold.

SVG. Scalable Vector Graphics, the format most icons are served in; it stays sharp at any size.

Webfont icons. Icons delivered as characters of a font, as with Material and Remix's webfont.

SERVICES > ICON CDN

Limits and accuracy

What the Icon CDN's names, categories, search and counts can and cannot tell you, and its practical limits.

Names and search

- Names are the sets' own. The same idea can have different names in different sets (**house** in Lucide, **home** in Tabler), and the same name can mean slightly different drawings.
- A name without a set, or a name the named set lacks, resolves to the first set that has it. Name the set, and copy names from the Icon Library, to be sure what you get.
- Search matches names, ids and each icon's search terms. It does not understand synonyms beyond those terms.

Categories

Categories come from the sets that provide them. Almost half the icons (6,113) have no category, so browsing by category shows only part of the catalogue.

Counts

- The catalogue counts 12,896 icons and 32,803 variants. An icon counts once however many styles it has.
 - The licence reference counts SVG files in the upstream folders (21,736) differently from the catalogue, which merges variants into icons.
-

Markup

- Served SVGs are normalised: their own width and height are removed, the `viewBox` is kept, and colours use `currentColor`. Size them yourself.
 - Stroke width applies only to stroke-drawn icons.
 - Material icons depend on their webfont loading; until it does, they may show as plain characters.
-

The loader

- It fills placeholders once, when the page loads; call `inject()` for later ones.
 - It downloads the catalogue index (about 1.9 MB) and one markup file per set and style you use, on first use.
 - It does not report a missing icon as an error; the placeholder stays as it was.
-

Related pages

- [Use the loader](#)
- [Collections](#)

SERVICES > ICON CDN

Privacy

What a page that uses the Icon CDN requests from it, and what the loader script does and does not do.

What your visitors' browsers request

You use	Your visitors' browsers request from icons.auricartisan.com
The loader with placeholders	The loader, the index, the set list, one markup file per set and style, and Material data and fonts if used

You use	Your visitors' browsers request from icons.auricartisan.com
Linked SVG files	Each file
Inline SVG, JSX, data URLs or a sprite you pasted	Nothing

As with any file host, the CDN receives the ordinary information that comes with each request.

What the loader does not do

The loader served at <https://icons.auricartisan.com/icon-cdn.js>:

- sets no cookies;
- writes nothing to the browser's storage;
- makes no requests other than the data and font files described above.

Avoiding the CDN at run time

If you prefer your visitors' browsers never to contact the CDN, paste inline SVG or a sprite, or export a ZIP and serve the files yourself. See [Export and self-host](#).

The Icon Library page

Recolouring, resizing and exports on the Icon Library page happen in your browser, and your selection is "Kept in this browser". For the auricartisan.com website as a whole, see the site's privacy policy at <https://auricartisan.com/legal/privacy/>.

SERVICES > ICON CDN

Related

The Icon Library page, VS Code icon extensions, the Font CDN and other related Auric Artisan products.

Product	Why	Docs
Icon Library (website)	Browse, recolour, resize and copy icons; export ZIPs and sprites	Icon Library
Auric Artisan Icon Hub (VS Code)	Work with icons from inside VS Code	Icon Hub
Font CDN	Fonts served the same way, from fonts.auricartisan.com	Font CDN
Public API	Colour and accessibility checks for your code	Public API

Links:

- Icon Library: <https://auricartisan.com/tool/icon/>

- Licence reference: <https://icons.auricartisan.com/LICENSE.md>

SERVICES > PUBLIC API

Auric Artisan REST API

What the Auric Artisan REST API is, who it is for, where to find it, and how to make a first call.

The Auric Artisan REST API exposes the platform's colour-science toolkit as JSON over HTTPS: colour conversions across 63 colour spaces, palettes, harmonies, gradients, shade scales and design tokens, WCAG and APCA accessibility checks, colour-vision deficiency simulation, Ishihara plates, a URL analyzer, and large browsable collections of colours, palettes, harmonies, gradients, shade scales and accessible colour pairs. Every endpoint lives under `/v1`.

You authenticate with an API key you create in your Auric Artisan account dashboard. Usage is metered in **credits**: one credit for a standard request, more for heavier work such as rendering images. API access is included from the **Specialist** plan upward, with a monthly credit allowance, a per-minute rate limit and optional prepaid overage. Discovery endpoints, such as the index, the catalogue and the OpenAPI document, are open to everyone so you can evaluate the API before buying.

The API is documented on the site at <https://auricartisan.com/library/documentation/api/>, with a separate guide to the analyzer endpoints at <https://auricartisan.com/library/documentation/api/analyzer/>. The API also serves its own interactive reference and OpenAPI 3.1 document.

Who it is for

- **Developers** building colour or accessibility features into products, design tools, CMSs or build pipelines.
- **Design-system teams** generating and checking tokens, scales and palettes automatically.
- **Accessibility and QA teams** checking colour pairs and watching many pages for metadata, security-header and markup regressions.
- **Data and research users** browsing the colour, palette and contrast collections with citable record ids.

Where to find it

What	Where
Base URL	https://api.auricartisan.com
Documentation	https://auricartisan.com/library/documentation/api/
Analyzer API guide	https://auricartisan.com/library/documentation/api/analyzer/
Interactive reference	https://api.auricartisan.com/v1/docs
OpenAPI 3.1 document	https://api.auricartisan.com/v1/openapi.yaml
API keys	Your account dashboard, https://auricartisan.com/auth/dashboard/

Quick start

1. Make sure your account is on a plan that includes API access: **Specialist** or **Industrial Pro**.
2. Open your dashboard, find **Your API Keys**, and choose **Create your first key** (or **Create key**).
3. Name the key, for example "CI pipeline", and choose **Create**.
4. Copy the key straight away. "This is the only time the full key is shown."
5. Send it in the `x-api-key` header with a request:

```
curl -X POST https://api.auricartisan.com/v1/color/convert \  
-H "x-api-key: aa_YOUR_KEY" \  
-H "Content-Type: application/json" \  
-d '{"color": "#d3af37", "to": "oklch"}'
```

6. Read the JSON response, and the `x-credits-cost` and `x-quota-remaining` headers that say what the call cost and what you have left.
7. Call `GET /v1` or `GET /v1/catalog` to see everything else the API can do.

What you can do

- Convert colours between common spaces, or any of 63 registry spaces with white-point control.
- Measure perceptual colour difference (CIEDE2000, CIE94, CIE76, CMC) and correlated colour temperature.
- Generate, score and export palettes, harmonies, gradients and shade scales, and render them as PNG.
- Turn one seed colour into a complete, contrast-checked design-token system and export it for ten platforms.
- Check foreground and background pairs against WCAG 2.2 and APCA, get the nearest accessible alternative, map where accessible colours lie on a background, and audit a palette.
- Simulate colour-vision deficiencies for colours, palettes and images, and find pairs that stop being distinguishable.
- Generate Ishihara test plates.
- Audit a web page's markup for SEO, metadata, security headers, sitemap and responsive signals, and render the report as HTML or PDF.
- Browse the colour atlas and the palette, harmony, gradient, shade and accessibility collections.
- Track usage, run long jobs asynchronously, subscribe to webhooks, and query everything through GraphQL.

In this folder

- [docs/](#)
 - [Getting started](#)

- Authentication
- Plans, credits and quotas
- Endpoints
- Request examples
- Errors
- SDKs and tooling
- Reference
- Troubleshooting
- FAQ
- others/
 - Glossary
 - Limits and accuracy
 - Related
 - Release notes
 - Privacy

SERVICES > PUBLIC API

Public API documentation

The contents of the public API documentation, from creating a key to every endpoint, error and limit.

This folder explains how to get access to the Auric Artisan REST API, authenticate, understand what calls cost, and use each group of endpoints. The [Reference](#) lists every endpoint in one table.

Files

Page	What it covers
Getting started	Get a key, make a first call, read the response and its headers
Authentication	API keys, the OAuth bearer token, key safety and revocation
Plans, credits and quotas	Plans, rate limits, credits per endpoint, overage, spend caps and ceilings, usage headers
Request examples	Ready-to-run requests for common tasks
Errors	The error format, every error code and what to do about it
SDKs and tooling	Official clients and integrations
Reference	Every endpoint with its method, authentication, credits and purpose
Troubleshooting	Common problems and fixes
FAQ	Short answers

Folders

Folder	What it covers
endpoints/	One page per group of endpoints, with parameters, responses and examples

SERVICES > PUBLIC API

Getting started

Get an API key from your dashboard, make a first request, and read the response and its usage headers.

This page takes you from no key to a working request.

1. Check your plan

The API is a paid feature, included from the **Specialist** plan upward:

Plan	API access
Apprentice (free)	Not included
Artisan	Not included
Specialist	Included
Industrial Pro	Included

A key created on a plan without API access authenticates, but metered endpoints refuse it with **402 PLAN_UPGRADE_REQUIRED**, and the response names the upgrade path. In the dashboard, key creation says "Available from the Specialist plan" until you upgrade.

You can still explore without a key. These discovery endpoints are open to everyone: **GET /v1**, **GET /v1/catalog**, **GET /v1/openapi.yaml**, **GET /v1/docs**, **GET /v1/tiers** and **GET /v1/health**.

2. Create a key

1. Sign in and open your dashboard at <https://auricartisan.com/auth/dashboard/>. The API documentation calls this **Dashboard > API Keys**.
2. Under **Your API Keys**, choose **Create your first key** (or **Create key** if you already have one).
3. Type a name in **Key name (e.g. "CI pipeline")** and choose **Create**.
4. Copy the key and store it somewhere safe, such as a password manager or your deployment's secret store. "This is the only time the full key is shown."

Keys start with **aa_**. You can hold up to 10 active keys, and **Revoke** any of them instantly.

3. Make a first request

Send the key in the **x-api-key** header:

```
# Convert a color to OKLCH
curl -X POST https://api.auricartisan.com/v1/color/convert \
  -H "x-api-key: aa_YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{"color": "#d3af37", "to": "oklch"}'
```

The response is JSON:

```
{"input":{"rgb":[211,175,55],"hex":"#d3af37"},"to":"oklch",
  "value":[0.7655,0.1384,91.55]}
```

Tool endpoints accept JSON through **POST** and reply in JSON. Collection and discovery endpoints use **GET** with query parameters.

4. Read the headers

Every response carries your live consumption, so a client can throttle itself:

```
x-ratelimit-limit: 300           # requests allowed per minute
x-ratelimit-remaining: 297
x-ratelimit-reset: 1783502302    # unix seconds
x-quota-limit: 1000000           # included credits this month
x-quota-used: 412
x-quota-remaining: 999588
x-quota-reset: 1785091200        # unix seconds
x-credits-cost: 10               # what this call consumed
```

See [Plans](#), [credits](#) and [quotas](#) for what each one means.

5. Check your usage

```
curl https://api.auricartisan.com/v1/usage/me -H "x-api-key: aa_YOUR_KEY"
```

```
{"tier": "pro", "plan": "specialist", "auth_type": "api_key",
 "limits": {"rate_per_minute": 300, "monthly_quota": 1000000},
 "this_month": {"slot": "2026-08", "used": 412}}
```

`used` counts credits, not calls.

6. Explore

- `GET /v1` lists every endpoint with the current version.
- `GET /v1/catalog` returns every endpoint with its parameters and examples, as JSON.
- `GET /v1/docs` is the interactive reference; `GET /v1/openapi.yaml` is the OpenAPI 3.1 document.
- `GET /v1/methods/:tool` lists the algorithms a tool supports, for example `/v1/methods/harmony`.
- `GET /v1/formats` lists every export format.

Next steps

- [Endpoints](#) by group
- [Request examples](#)
- [Errors](#)

SERVICES > PUBLIC API

Authentication

How to authenticate with the Auric Artisan API — API keys, the CLI's OAuth bearer token, keeping keys safe, and revoking them.

Every metered endpoint requires a credential. Requests without a valid one receive **401 Unauthorized**. Endpoints marked "None" in the [Reference](#) are public and unmetered.

Methods

Method	How to send it	Best for
API key (recommended)	The <code>x-api-key: aa_...</code> header, or the <code>?key=aa_...</code> query parameter	Servers, scripts and CI; create keys in the dashboard
OAuth bearer	<code>Authorization: Bearer <token></code> , from the CLI's device flow	The <code>auric</code> command-line tool and interactive tools

Prefer the header. A key in a query string can end up in server logs and browser history.

```
curl https://api.auricartisan.com/v1/usage/me -H "x-api-key: aa_YOUR_KEY"
```

Keys are secret

- The full key is displayed **once**, when you create it. It is stored only as a SHA-256 hash, so nobody, including Auric Artisan, can recover it later. If you lose it, create a new one.
- Keep keys out of source code and front-end code. Load them from environment variables or a secret store, as the examples do with `$AURIC_API_KEY`.
- Each successful request updates the key's "last used" time, which you can check in the dashboard to spot a key in use where it should not be.

Several keys

You can hold up to **10** active keys, for example one per service or environment. Rate limits, monthly credits and the credit balance are pooled **per account**, not per key: ten keys share one allowance, so splitting traffic across keys does not raise your limits.

Revoke a key

If a key leaks, revoke it in the dashboard with **Revoke**. Revocation takes effect within a minute. Requests with a revoked key then receive **401**.

Plan and access

A valid key on a plan without API access authenticates, but metered endpoints refuse it with **402 PLAN_UPGRADE_REQUIRED**. See [Plans, credits and quotas](#).

Related pages

- [Getting started](#)
- [Errors](#)
- The account documentation, in the website section of these docs, covers the dashboard itself: [Account and billing](#)

SERVICES > PUBLIC API

Errors

The Auric Artisan API's error format, every error code with its HTTP status, what each means, and how to respond.

Errors are JSON with a stable **code** and a human-readable **message**. Most include a **request_id**; quote it when you contact support about a specific call.

```
{
  "error": {
    "code": "INVALID_INPUT",
    "message": "seed: not a number"
  },
  "request_id": "daebcbcd-508b-4d4d-91ef-87ee99d32f03"
}
```

Error codes

Status	Code	Meaning and what to do
400	INVALID_INPUT	A parameter is missing or malformed; the message names the field
400	INVALID_JSON	The body is not valid JSON
401	UNAUTHORIZED or UNAUTHENTICATED	No key, or a missing, invalid or revoked key
402	PLAN_UPGRADE_REQUIRED	A valid key on a plan without API access; included from Specialist upward
402	INSUFFICIENT_CREDITS	Overage is on but the prepaid balance will not cover the call. Top up.
402	SPEND_CAP_REACHED	Your monthly overage spend cap was reached. Raise it in the dashboard.

Status	Code	Meaning and what to do
402	CREDIT_DEBIT_FAILED	The balance could not be debited, so the request was not served and nothing was charged. Retry.
403	FORBIDDEN	A valid key without rights for this route
404	NOT_FOUND	Unknown endpoint or resource id. <code>GET /v1</code> lists every endpoint.
405	METHOD_NOT_ALLOWED	Right path, wrong method; the <code>Allow</code> header lists what is accepted
413	PAYLOAD_TOO_LARGE	The body is over the limit (4 MB for image endpoints)
429	RATE_LIMITED	Per-minute rate exceeded. Wait for <code>Retry-After</code> .
429	IP_RATE_LIMITED	Too many requests from one address, whatever the key
429	QUOTA_EXCEEDED	Included monthly credits used up and overage off. Enable it, or wait for the reset.
429	HARD_CEILING_REACHED	Ten times the included allowance was reached this month; contact support
500	INTERNAL	An unexpected server error. Quote the <code>request_id</code> .
503	NOT_CONFIGURED	An optional part of the service is not enabled
503	ENFORCEMENT_UNAVAILABLE	Limits could not be checked, so nothing was executed. Retry later.

The API reference lists the 401 code as both `UNAUTHENTICATED` (in the endpoint tables) and `UNAUTHORIZED` (in the errors section); handle 401 by status rather than by code.

Some endpoints have their own codes, such as `UNKNOWN_SPACE` (404) for `/v1/science/spaces/:key`, `UNKNOWN_CONDITION` for `/v1/vision/conditions/:slug`, `VALIDATION_ERROR` for an unknown collection id, and the analyzer's codes on [Analyzer](#).

429 or 402: which is which

The split is by what you should do:

- **429** means back off and retry. The condition resets on its own, at the time in `retry-after` and `details.resets_at`. Official SDKs already retry on it.
- **402** means only a money decision unblocks it: a top-up, a raised spend cap or a plan upgrade. Retrying without one is pointless.

Every refusal names its remedy in `details.remedy`: `enable_overage`, `buy_credits`, `raise_spend_cap` or `contact_support`, alongside `credits_used`, `credits_included`, `credits_this_request` and `resets_at`.

```
{
  "error": {
    "code": "INSUFFICIENT_CREDITS",
    "message": "Pay-as-you-go is on, but the prepaid credit balance is empty. Top up to continue – nothing is billed after the fact.",
    "details": {
      "tier": "pro",
      "credits_used": 1000010,
      "credits_included": 1000000,
      "credits_this_request": 10,
      "resets_at": "2026-09-01T00:00:00.000Z",
      "remedy": "buy_credits",
      "credits_needed": 10
    }
  },
  "request_id": "daebcbcd-508b-4d4d-91ef-87ee99d32f03"
}
```

Handling errors well

- Branch on `error.code`, not on `message`, which may change.
- On 429, wait for `retry-after` before retrying; do not retry in a tight loop.
- On 402, log `details.remedy` so someone can act on it.
- Watch `x-quota-remaining` to slow down before you run out.

Related pages

- [Plans, credits and quotas](#)
- [Troubleshooting](#)

SERVICES > PUBLIC API

Request examples

Ready-to-run requests for common tasks with the Auric Artisan API.

Replace `aa_YOUR_KEY` with your key, or set `AURIC_API_KEY` in your shell and use `$AURIC_API_KEY`.

Convert a colour to OKLCH

```
curl -X POST https://api.auricartisan.com/v1/color/convert \
  -H "x-api-key: aa_YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{"color": "#d3af37", "to": "oklch"}'
# → {"input":{"rgb":[211,175,55],"hex":"#d3af37"},"to":"oklch",
#    "value":[0.7655,0.1384,91.55]}
```

Check contrast

```
curl -X POST https://api.auricartisan.com/v1/color/contrast \
  -H "x-api-key: aa_YOUR_KEY" -H "Content-Type: application/json" \
  -d '{"fg": "#1a1712", "bg": "#d3af37"}'
```

For WCAG 2.2 and APCA together:

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/check" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"fg": "#767676", "bg": "#ffffff"}'
```

Fix a failing pair

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/recommend" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"fg": "#c9a227", "bg": "#ffffff", "target": "foreground", "minRatio": 4.5}'
```

Generate a palette

```
curl -X POST https://api.auricartisan.com/v1/palette/generate \
  -H "x-api-key: aa_YOUR_KEY" -H "Content-Type: application/json" \
  -d '{"method": "analogous", "count": 5, "hue": 45}'
```

Simulate colour-vision deficiency

```
curl -X POST "https://api.auricartisan.com/v1/vision/simulate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"color": "#c9a227", "model": "brettel-deutan", "severity": 1}'
```

model takes a model id from `GET /v1/vision/models`. To work with condition names such as **protanopia**, use `/v1/vision/palette`:

```
curl -X POST "https://api.auricartisan.com/v1/vision/palette" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"colors": ["#D3AF37", "#2E8B57"], "conditions": ["deuteranopia", "tritanopia"]}'
```

Build a token system and export it

```
curl -X POST "https://api.auricartisan.com/v1/tokens/generate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"seed":"#c9a227","name":"Auric","harmony":"analogous"}'

curl -X POST "https://api.auricartisan.com/v1/tokens/export" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"format":"css","tokens":[{"name":"--color-primary","value":"#c9a227"}]}'
```

Audit a page's metadata

```
curl -X POST "https://api.auricartisan.com/v1/analyzer/inspect" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"url": "https://example.com/", "scope": ["seo"] }'
```

Check your usage

```
curl https://api.auricartisan.com/v1/usage/me -H "x-api-key: aa_YOUR_KEY"
# → {"tier":"pro","plan":"specialist","auth_type":"api_key",
#    "limits":{"rate_per_minute":300,"monthly_quota":1000000},
#    "this_month":{"slot":"2026-08","used":412}}
#
# `used` counts credits, not calls.
```

From JavaScript

```
const res = await fetch("https://api.auricartisan.com/v1/color/contrast", {
  method: "POST",
  headers: { "x-api-key": process.env.AURIC_API_KEY, "content-type": "application/json" },
  body: JSON.stringify({ fg: "#767676", bg: "#ffffff" }),
});
if (!res.ok) throw new Error((await res.json()).error.code);
const { ratio, aa, aaa, aa_large } = await res.json();
console.log(ratio, aa, aaa, aa_large, res.headers.get("x-credits-cost"));
```

Related pages

- [Endpoints](#)
- [SDKs and tooling](#)

SERVICES > PUBLIC API

Plans, credits and quotas

What each plan includes, how credits are charged per endpoint, rate limits, overage, spend caps, the hard ceiling, and the headers that report usage.

Limits follow your account plan. The rate limit, monthly credits and credit balance are pooled per account, not per key.

What each plan includes

Plan	API tier	Requests per minute	Credits per month	Hard ceiling	Max keys
Apprentice (free)	Free	—	Not included	—	—
Artisan	Paid	—	Not included	—	—
Specialist	Pro	300	1,000,000	10,000,000	10
Industrial Pro	Enterprise	3,000	10,000,000	100,000,000	10

GET /v1/tiers returns the machine-readable version of this table, so a client can read its own terms instead of hard-coding them.

Note: Industrial Pro was sold as "Unlimited" before 12 August 2026. Subscriptions created before that date keep that promise: they are metered at 1,000,000,000 credits a month, a hundred times the new figure, and their hard ceiling scales with that allowance. The 10,000,000 figure applies to subscriptions started on or after 12 August 2026.

Credits

The monthly quota is counted in **credits**, not calls. One credit is one standard request; endpoints that do more work cost more, so simple colour maths does not subsidise image rendering. Every response states its cost in the **x-credits-cost** header.

- **1 credit:** Standard work: `/v1/color/convert`, `/v1/color/contrast`, `/v1/science/convert`, `/v1/accessibility/check`, `/v1/vision/simulate`, and anything not listed below
- **2–4 credits:** Generators and multi-step maths: `/v1/palette/generate`, `/v1/harmony/generate`, `/v1/gradient/generate`, `/v1/shade/scale`, `/v1/science/delta-e`, `/v1/science/temperature`, `/v1/tokens/generate`, `/v1/graphql`
- **5 credits:** `/v1/science/dossier`, and `/v1/usage/stream` (charged once when the stream opens; events are free)
- **10–12 credits:** Rendered pixels: `/v1/palette/png`, `/v1/gradient/png`, `/v1/ishihara/png`, `/v1/vision/simulate-image`, `/v1/palette/from-image`, `/v1/accessibility/from-image`

- **25 credits:** Async jobs: `POST /v1/jobs/{type}`. Polling a job with `GET /v1/jobs/id/{id}` costs 1.

The analyzer endpoints cost: `GET /v1/analyzer/stages` 1 credit, `POST /v1/analyzer/inspect` 12 credits and `POST /v1/analyzer/render` 10 credits. See [Analyzer](#).

A new endpoint that has not been given its own weight bills as standard work, 1 credit; nothing is free by accident. Public discovery endpoints are unmetered.

When you run out

By default, running out of included credits stops the traffic. Further calls return `429 QUOTA_EXCEEDED` until the monthly reset. Nothing is charged.

Pay-as-you-go overage (opt-in)

- Overage is **off** until you turn it on in the dashboard's API keys area.
- It is **prepaid**: it spends a credit balance bought in advance, at **\$1.00 (₹85) per 100,000 credits**.
- If the balance will not cover a call, the call is refused with `402 INSUFFICIENT_CREDITS`. It is never served on account, and no usage produces an invoice after the fact.
- Buy credit packs in the dashboard with **Add credits**. Purchased credits do not expire at the end of the period.

In the dashboard, the overage switch reads "Off — calls stop at your included allowance. Nothing is charged." while it is off, and asks you to confirm with **Turn on paid overage** or **Keep it off**.

Spend cap

Set a **Monthly spend cap** in money, up to \$10,000. Overage stops there with `402 SPEND_CAP_REACHED`; raise the cap in the dashboard to continue.

Hard ceiling

Independent of anything you configure, no account may spend more than **ten times its included allowance** in one month. Reaching it returns `429 HARD_CEILING_REACHED`. It is a safety stop against runaway retries and leaked keys; support can raise it.

Reading the meter

Every response carries these headers:

Header	Meaning
<code>x-ratelimit-limit</code>	Requests allowed per minute
<code>x-ratelimit-remaining</code>	Requests left this minute
<code>x-ratelimit-reset</code>	When the minute resets, in Unix seconds

Header	Meaning
<code>x-quota-limit</code>	Included credits this month
<code>x-quota-used</code>	Credits used this month
<code>x-quota-remaining</code>	Included credits left
<code>x-quota-reset</code>	When the monthly quota resets, in Unix seconds
<code>x-credits-cost</code>	What this call consumed
<code>x-credits-overage</code>	Only when the call was paid from the prepaid balance: the overage credits used
<code>x-overage-cost-usd</code>	Only with overage: 1.00 when this call opened a new 100,000-credit block, otherwise 0.00
<code>x-credits-ceiling</code>	The hard ceiling

Check usage at any time with `GET /v1/usage/me`, or full per-day, per-endpoint history with `GET /v1/usage`. A `quota.threshold` webhook fires when you cross 80% of the included allowance.

The dashboard

The dashboard's API area shows **Usage & cost** (credits used, included credits left, overage credits and overage cost, and **Credits per day**), your **prepaid credits** balance, **Spend controls**, **Recent credit activity**, **Where the credits went**, and a live feed of recent calls once you connect a key.

Beyond the largest plan

For volume beyond the largest credit pack, a custom rate limit or a contract, contact Auric Artisan. Billing terms for prepaid credits, including expiry, cancellation, downgrade and refunds, are in the Subscription & Billing Policy at <https://auricartisan.com/legal/subscription-billing/>.

Related pages

- [Errors](#), for the 429 and 402 codes
- [Usage](#), [jobs](#), [webhooks](#) and [GraphQL](#)

SERVICES > PUBLIC API

SDKs and tooling

The official clients and integrations for the Auric Artisan API — JavaScript, Python, the CLI, an MCP server, a Postman collection and GraphQL.

The API documentation lists these official clients and integrations. All of them use the same `/v1` endpoints.

Client	Package or location	Notes
JavaScript	<code>@auric-artisan/api</code>	Node 18 and later, browsers and Workers. Typed methods for every endpoint.
Python	<code>auric-artisan</code>	Python 3.8 and later, standard library only, no dependencies
CLI	<code>@auric-artisan/cli</code>	For example <code>auric contrast '#222' '#fff'</code> in your terminal
MCP server	Listed on the API page	Gives Claude, Cursor or any MCP host direct access to the colour tools
Postman	Listed on the API page	A ready-made collection covering the whole API
GraphQL	<code>POST /v1/graphql</code>	A single-endpoint alternative, with a playground at <code>GET /v1/graphql</code>

Note: The documentation names these packages; check your package registry for their availability and current versions before depending on them.

The CLI and OAuth

The `auric` CLI signs in with a device flow and sends an OAuth bearer token (`Authorization: Bearer <token>`) instead of an API key. See [Authentication](#).

Generating your own client

`GET /v1/openapi.yaml` returns the OpenAPI 3.1 document, which most API tools can import or use to generate a client. `GET /v1/catalog` returns the same information as JSON, with parameters and examples.

Retries

The official SDKs retry automatically on `429` responses. If you write your own client, honour `retry-after`; see [Errors](#).

Related pages

- [Getting started](#)
- [Request examples](#)

SERVICES > PUBLIC API

Reference

Every customer endpoint of the Auric Artisan REST API with its method, authentication, credit cost and purpose, plus the headers, input formats, plans and limits.

Essentials

Item	Value
Base URL	<code>https://api.auricartisan.com</code>
Version prefix	<code>/v1</code>
Format	JSON over HTTPS
Authentication	<code>x-api-key</code> header (or <code>?key=</code> query), or <code>Authorization: Bearer</code> from the CLI
Key prefix	<code>aa_</code>
Keys per account	Up to 10 active
Revocation	Takes effect within a minute
Specification	<code>GET /v1/openapi.yaml</code> (OpenAPI 3.1); <code>GET /v1/catalog</code> (JSON)
Interactive reference	<code>GET /v1/docs</code>

Colour input

Unless a field says otherwise, a colour can be sent as `#rrggbb`, `#rgb`, `rgb(r,g,b)`, an array `[r,g,b]` or an object `{r,g,b}`. Image fields take Base64 or a data URL.

Endpoints

Auth "None" means public and unmetered; "Key" needs an API key or bearer token. Credits are per call; see [Plans, credits and quotas](#). The documentation gives some costs as ranges (2–4, 10–12); the exact cost of every call is returned in `x-credits-cost`. Endpoints the documentation does not weight separately bill as standard work, 1 credit.

Method	Path	Auth	Credits	Summary
GET	<code>/v1</code>	None	—	Index of every endpoint, with the current version
GET	<code>/v1/health</code>	None	—	Liveness probe; always cheap, never touches storage
GET	<code>/v1/ready</code>	None	—	Readiness probe: whether each backing store is bound
GET	<code>/v1/methods/:tool</code>	Key	1	The algorithms a given tool supports
GET	<code>/v1/formats</code>	None	—	Every export format, across every exportable resource
GET	<code>/v1/tiers</code>	None	—	Rate limit and monthly quota for each tier
GET	<code>/v1/metrics</code>	None	—	Process-level counters in Prometheus format

Method	Path	Auth	Credits	Summary
GET	<code>/v1/status</code>	None	—	Human-readable status page
GET	<code>/v1/catalog</code>	None	—	Every endpoint with its parameters and examples, as JSON
GET	<code>/v1/docs</code>	None	—	Interactive reference (Redoc)
GET	<code>/v1/openapi.yaml</code>	None	—	The OpenAPI 3.1 document
GET	<code>/v1/graphql</code>	None	—	GraphQL playground
POST	<code>/v1/graphql</code>	Key	2–4	Run a GraphQL query against the same resolvers as the REST endpoints
POST	<code>/v1/color/convert</code>	Key	1	Convert one colour between the common spaces
GET	<code>/v1/color/atlas</code>	Key	1	Browse the 8,192-colour atlas under a filter
GET	<code>/v1/color/atlas/stats</code>	Key	1	What the atlas actually contains
GET	<code>/v1/color/atlas/:id</code>	Key	1	Resolve one sample, whole
POST	<code>/v1/color/nearest</code>	Key	1	The catalogued colours closest to one you already have
POST	<code>/v1/color/contrast</code>	Key	1	WCAG contrast ratio between two colours, with pass or fail at each level
GET	<code>/v1/science/spaces</code>	Key	1	Every colour space the platform knows, with provenance
GET	<code>/v1/science/spaces/:key</code>	Key	1	One colour space in full, including its conversion matrices
POST	<code>/v1/science/convert</code>	Key	1	Convert a colour into any registry space, or several at once
POST	<code>/v1/science/dossier</code>	Key	5	One colour in every space at once, plus its colour temperature
POST	<code>/v1/science/delta-e</code>	Key	2–4	Perceptual difference between two colours
POST	<code>/v1/science/temperature</code>	Key	2–4	Correlated colour temperature from a colour, or the colour of a temperature
GET	<code>/v1/science/illuminants</code>	Key	1	Standard reference illuminants with their chromaticities
GET	<code>/v1/science/metrics</code>	Key	1	The difference formulae, their options, and when to use each
POST	<code>/v1/palette/generate</code>	Key	2–4	Generate a palette from a seed colour
POST	<code>/v1/palette/evaluate</code>	Key	1	Score a palette for harmony, contrast and accessibility
POST	<code>/v1/palette/export</code>	Key	1	Render a palette into a file format

Method	Path	Auth	Credits	Summary
POST	<code>/v1/palette/from-image</code>	Key	10–12	Extract a palette from an uploaded image
POST	<code>/v1/palette/png</code>	Key	10–12	Render a palette as a PNG swatch sheet
POST	<code>/v1/harmony/generate</code>	Key	2–4	Build a harmony set from a base colour
POST	<code>/v1/harmony/classify</code>	Key	1	Identify which harmony an existing set of colours follows
POST	<code>/v1/harmony/export</code>	Key	1	Export a harmony set
POST	<code>/v1/gradient/generate</code>	Key	2–4	Interpolate a gradient between stops, in a chosen space
GET	<code>/v1/shade/library</code>	Key	1	Browse 8,192 measured shade scales under a filter
GET	<code>/v1/shade/library/stats</code>	Key	1	What the shade collection contains
GET	<code>/v1/shade/library/:id</code>	Key	1	One scale, whole, with every measurement
POST	<code>/v1/shade/evaluate</code>	Key	1	Measure a scale you already have: does it work as a token ramp?
GET	<code>/v1/palette/library</code>	Key	1	Browse five million generated palettes under a filter
GET	<code>/v1/palette/library/stats</code>	Key	1	What the palette collection looks like, measured on a stated sample
GET	<code>/v1/palette/library/:id</code>	Key	1	One palette, whole, with the derivation that reproduces it
GET	<code>/v1/harmony/library</code>	Key	1	Browse the 8,192-harmony collection under a filter
GET	<code>/v1/harmony/library/stats</code>	Key	1	What the harmony collection contains
GET	<code>/v1/harmony/library/:id</code>	Key	1	One harmony, whole, with the derivation that reproduces it
POST	<code>/v1/harmony/identify</code>	Key	1	Name the scheme your colours form
GET	<code>/v1/gradient/library</code>	Key	1	Browse the 8,192-gradient collection under a filter
GET	<code>/v1/gradient/library/stats</code>	Key	1	What the gradient collection contains
GET	<code>/v1/gradient/library/:id</code>	Key	1	One gradient, whole
POST	<code>/v1/gradient/analyse</code>	Key	1	Measure a gradient you already have: will it band, and where?
POST	<code>/v1/gradient/export</code>	Key	1	Export a gradient as CSS, SVG or a design-tool format
POST	<code>/v1/gradient/png</code>	Key	10–12	Render a gradient as a PNG
POST	<code>/v1/shade/scale</code>	Key	2–4	Build a tint and shade scale from one colour
POST	<code>/v1/shade/export</code>	Key	1	Export a shade scale

Method	Path	Auth	Credits	Summary
POST	<code>/v1/tokens/generate</code>	Key	2–4	Turn one seed colour into a complete, contrast-checked token system
POST	<code>/v1/tokens/export</code>	Key	1	Render a token deck for a specific platform
GET	<code>/v1/tokens/formats</code>	Key	1	The ten token export targets
GET	<code>/v1/personalize/vocabulary</code>	Key	1	Everything a brief may name, with the ranges the engine applies
GET	<code>/v1/personalize/presets</code>	Key	1	The 17 briefs the tool ships, in request shape
POST	<code>/v1/personalize/pool</code>	Key	1	Grow a working set of colours from seeds and a brief
POST	<code>/v1/personalize/score</code>	Key	1	Rank a palette you already have on the search's six scores
POST	<code>/v1/personalize/palettes</code>	Key	1	Search: N candidates in, the best K out, each with its breakdown
POST	<code>/v1/personalize/system</code>	Key	1	One brief in, an entire design system out
POST	<code>/v1/accessibility/check</code>	Key	1	Check a foreground and background pair against WCAG 2.2 and APCA
POST	<code>/v1/accessibility/recommend</code>	Key	1	Suggest the nearest accessible alternative to a failing pair
POST	<code>/v1/accessibility/from-image</code>	Key	10–12	Audit the colour contrast present in an image
POST	<code>/v1/accessibility/map</code>	Key	1	The whole territory of foreground colour a background leaves you
POST	<code>/v1/accessibility/picks</code>	Key	1	A ready set of foregrounds that clear a target on one background
POST	<code>/v1/accessibility/audit</code>	Key	1	Audit a palette against one background, with a fix for each failure
GET	<code>/v1/accessibility/corpus</code>	Key	1	Browse the five-million-pair corpus under a filter, with a cursor
GET	<code>/v1/accessibility/corpus/stats</code>	Key	1	The measured distribution of contrast across the corpus
GET	<code>/v1/accessibility/corpus/:record_id</code>	Key	1	Resolve one record by id or index
GET	<code>/v1/analyzer/stages</code>	Key	1	What the analyzer can check, and how much markup alone can answer
POST	<code>/v1/analyzer/inspect</code>	Key	12	Audit one page from its markup; no browser, no rendering
POST	<code>/v1/analyzer/render</code>	Key	10	Turn a report into an HTML or PDF document
GET	<code>/v1/vision/models</code>	Key	1	The colour-vision deficiency models available

Method	Path	Auth	Credits	Summary
POST	<code>/v1/vision/simulate</code>	Key	1	Simulate how a colour appears under a vision deficiency
GET	<code>/v1/vision/conditions</code>	Key	1	Every documented vision condition, and what can simulate it
GET	<code>/v1/vision/conditions/:slug</code>	Key	1	One condition, with its severity, prevalence and a worked example
GET	<code>/v1/vision/matrices</code>	Key	1	The CVD matrices, for applying the transform yourself
POST	<code>/v1/vision/palette</code>	Key	1	Run a whole palette through many conditions in one call
POST	<code>/v1/vision/audit</code>	Key	1	Find which pairs in a palette stop being distinguishable
POST	<code>/v1/vision/simulate-image</code>	Key	10–12	Simulate a vision deficiency across a whole image
POST	<code>/v1/ishihara/plate</code>	Key	1	Generate an Ishihara test plate as structured data
POST	<code>/v1/ishihara/png</code>	Key	10–12	Render an Ishihara plate as a PNG
GET	<code>/v1/usage</code>	Key	1	Per-day, per-endpoint usage history
GET	<code>/v1/usage/me</code>	Key	1	Your tier, limits and month-to-date consumption
GET	<code>/v1/usage/live</code>	Key	1	The last 100 calls, with rolling latency and error stats
GET	<code>/v1/usage/stream</code>	Key	5	The same live feed over Server-Sent Events (charged once per stream)
GET	<code>/v1/jobs</code>	Key	1	The async job types available
POST	<code>/v1/jobs/:type</code>	Key	25	Queue a long-running job and get an id back immediately
GET	<code>/v1/jobs/id:id</code>	Key	1	Poll one job
GET	<code>/v1/jobs/id:id/stream</code>	Key	1	Follow a job to completion over SSE
GET	<code>/v1/webhooks/events</code>	None	—	Event types you can subscribe to
GET	<code>/v1/webhooks</code>	Key	1	List your webhook subscriptions
POST	<code>/v1/webhooks</code>	Key	1	Subscribe a URL to one or more events
GET	<code>/v1/webhooks/:id</code>	Key	1	Fetch one subscription
DELETE	<code>/v1/webhooks/:id</code>	Key	1	Revoke a subscription

The groups are explained, with parameters and examples, in [Endpoints](#).

Response headers

Header	Meaning
<code>x-ratelimit-limit</code>	Requests allowed per minute
<code>x-ratelimit-remaining</code>	Requests left this minute
<code>x-ratelimit-reset</code>	Minute reset, in Unix seconds
<code>x-quota-limit</code>	Included credits this month
<code>x-quota-used</code>	Credits used this month
<code>x-quota-remaining</code>	Included credits left
<code>x-quota-reset</code>	Monthly reset, in Unix seconds
<code>x-credits-cost</code>	Credits this call consumed
<code>x-credits-coverage</code>	Overage credits this call used (only when paid from the prepaid balance)
<code>x-coverage-cost-usd</code>	1.00 when the call opened a new 100,000-credit block, otherwise 0.00
<code>x-credits-ceiling</code>	The account's hard ceiling
<code>Retry-After</code>	With 429: when to retry
<code>Allow</code>	With 405: the accepted methods

Plans

Plan	API	Requests per minute	Credits per month	Hard ceiling	Max keys
Apprentice (free)	Not included	—	—	—	—
Artisan	Not included	—	—	—	—
Specialist	Included	300	1,000,000	10,000,000	10
Industrial Pro	Included	3,000	10,000,000	100,000,000	10

Overage is opt-in and prepaid at \$1.00 (₹85) per 100,000 credits, with a monthly spend cap of up to \$10,000.

Limits stated in the documentation

Limit	Value
Request body for image endpoints	4 MB
<code>/v1/usage/live</code>	The last 100 calls; <code>limit</code> 1–100
<code>/v1/usage/stream</code>	Heartbeat every 15 s; connections capped at 10 minutes

Limit	Value
<code>/v1/palette/generate</code>	1–64 colours
Vision palette and audit	Up to 64 colours
Accessibility audit	1–64 colours
Ishihara plate	64–1024 px; 50–5000 dots
Collection page size	Up to 200 (atlas, corpus) or 100 (libraries)
Corpus scan budget	<code>max_scan</code> up to 1,000,000
Palette library scan budget	<code>max_scan</code> 1,000–500,000

Related pages

- [Errors](#)
- [Endpoints](#)

SERVICES > PUBLIC API

Troubleshooting

Common problems with the Auric Artisan API — keys, plans, limits, credits, inputs, collections and the analyzer — and how to fix them.

Keys and access

Symptom	Why	What to do
<code>401</code> on every call	No key, a mistyped key, or a revoked key	Send the key in <code>x-api-key</code> ; check it starts with <code>aa_</code> ; create a new one if it was revoked or lost
<code>401</code> right after revoking a key	Revocation takes effect within a minute	Use another key
<code>402 PLAN_UPGRADE_REQUIRED</code>	Your plan does not include the API	Upgrade to Specialist or Industrial Pro
I cannot create a key in the dashboard	Key creation is available from the Specialist plan	Upgrade, or use the public discovery endpoints meanwhile
I lost a key	The full key is shown once and stored only as a hash	Create a new key and revoke the old one
<code>403 FORBIDDEN</code>	The key has no rights for this route	Check the path; use an endpoint listed in the Reference

Limits and credits

Symptom	Why	What to do
<code>429 RATE_LIMITED</code>	Too many requests this minute	Wait for <code>Retry-After</code> ; spread calls out
<code>429 IP_RATE_LIMITED</code>	Too many requests from one address	Slow down, whatever key you use
<code>429 QUOTA_EXCEEDED</code>	Included credits used up, overage off	Turn on prepaid overage and add credits, or wait for the reset in <code>details.resets_at</code>
<code>402 INSUFFICIENT_CREDITS</code>	Overage on, prepaid balance empty	Add credits in the dashboard
<code>402 SPEND_CAP_REACHED</code>	Your monthly spend cap was reached	Raise the cap in the dashboard
<code>429 HARD_CEILING_REACHED</code>	Ten times the included allowance used this month	Contact support
Adding more keys did not raise my limits	Limits are pooled per account	Upgrade the plan, or use overage
Credits drain faster than calls	Heavier endpoints cost more than 1 credit	Read <code>x-credits-cost</code> ; see Plans, credits and quotas

Requests

Symptom	Why	What to do
<code>400 INVALID_JSON</code>	The body is not valid JSON	Check quoting, especially in shell commands
<code>400 INVALID_INPUT</code>	A field is missing or malformed	Read the message, which names the field
<code>405 METHOD_NOT_ALLOWED</code>	Wrong method, such as <code>GET</code> for a <code>POST</code> endpoint	Use the method in the <code>Allow</code> header
<code>404 NOT_FOUND</code>	Unknown path or id	<code>GET /v1</code> lists every endpoint
<code>413 PAYLOAD_TOO_LARGE</code>	Image bodies are limited to 4 MB	Resize or compress the image
<code>/v1/vision/simulate</code> refuses <code>deuteranopia</code>	It takes model ids, not condition names	Use a model from <code>/v1/vision/models</code> , such as <code>brettel-deutan</code> , or use <code>/v1/vision/palette</code>
A colour is rejected	Unsupported format	Use <code>#rrggbb</code> , <code>#rgb</code> , <code>rgb(r,g,b)</code> , <code>[r,g,b]</code> or <code>{r,g,b}</code>

Collections

Symptom	Why	What to do
An empty page with a <code>next_cursor</code>	The scan budget ran out before any match (<code>truncated: true</code>)	Follow <code>next_cursor</code> , or raise <code>max_scan</code>
Sorted results look out of order across pages	In scanned collections, sorting other than by <code>id</code> is within each scan	Check <code>sorted_within_scan</code> ; sort on your side if you need a global order

Analyzer

Symptom	Why	What to do
<code>score</code> is always <code>null</code>	Deliberate: markup alone cannot measure every category	Use <code>category_scores</code> , or run the tool for an overall score
<code>400 RENDER_REQUIRED</code>	Contrast, accessibility, deep, performance, before-and-after and simulation need a browser	Remove those stages, or use the tool
<code>400 EMPTY_SCOPE</code>	<code>scope</code> was <code>[]</code>	Omit <code>scope</code> for everything, or name stages
<code>502 FETCH_FAILED</code>	The page could not be read: DNS, TLS, timeout, or a private address	Check the URL is public and reachable
Non-Latin text missing from the PDF	The PDF uses standard fonts that cover Latin characters	Use <code>format: "html"</code>

Getting help

Quote the `request_id` from the error response when you contact Auric Artisan about a specific call.

Related pages

- Errors
- FAQ

SERVICES > PUBLIC API

FAQ

Short answers to common questions about the Auric Artisan REST API.

Which plans include the API?

Specialist and Industrial Pro. Apprentice (free) and Artisan do not include it.

Can I try it before buying?

Yes, partly. The discovery endpoints are open without a key: `GET /v1`, `GET /v1/catalog`, `GET /v1/openapi.yaml`, `GET /v1/docs`, `GET /v1/tiers`, `GET /v1/health` and `GET /v1/webhooks/events`.

How many keys can I have?

Up to 10 active keys. They share one allowance, because limits are pooled per account.

What is a credit?

The unit the monthly quota is counted in. A standard request costs 1 credit; heavier work costs more, up to 25 for an async job. Each response says what it cost in `x-credits-cost`.

What happens when I run out?

Calls return `429 QUOTA_EXCEEDED` until the monthly reset, and nothing is charged, unless you turned on prepaid overage.

Can I be billed unexpectedly?

No. Overage is off by default, spends only credits bought in advance, stops at a spend cap you set, and stops unconditionally at ten times your included allowance. Nothing is invoiced after the fact.

Do purchased credits expire?

Purchased credits do not expire at period end. Full terms are in the Subscription & Billing Policy.

Is the API the same as the website tools?

It uses the same engines where it can, but some things differ. The analyzer endpoints read a page's source and refuse checks that need a browser, such as contrast; the website's Analyzer runs pages in your browser. API credits and the website's tool tokens are separate balances.

Why does the analyzer return no overall score?

Because a markup-only run cannot measure every category, and averaging a subset would give a misleading number. You get the measured categories in `category_scores` and a list of `unscored_categories`.

Which standards does the accessibility check support?

`/v1/accessibility/check` accepts `WCAG 2.1`, `WCAG 2.2`, `WCAG 3.0`, `Auric SD Flexible`, `Auric SD Strict`, or `all`, the default.

Is there a client library?

The documentation lists a JavaScript package (`@auric-artisan/api`), a Python package (`auric-artisan`), a CLI (`@auric-artisan/cli`), an MCP server, a Postman collection and GraphQL. See [SDKs and tooling](#).

Can I call the API from a web page?

Only with care: a key in front-end code is visible to anyone. Call the API from your server and keep the key secret.

How do I cite a colour pair or palette?

Use its record id, such as `acc_2j` or `pal_1z`. Records are derived from a published seed, so anyone can recover the same record with the matching `/:id` endpoint.

SERVICES > PUBLIC API

Endpoints

The groups of endpoints in the Auric Artisan REST API, with a page for each group.

Everything lives under `/v1` on <https://api.auricartisan.com>. Each page below covers one group of endpoints: what each does, its parameters, what it returns and an example. The [Reference](#) lists them all in one table.

Conventions

- **Colours** in request bodies can be written as `#rrggbb`, `#rgb`, `rgb(r,g,b)`, an array `[r,g,b]` or an object `{r,g,b}`, unless a field says otherwise.
- **Tool endpoints** take JSON through `POST`, with `content-type: application/json`.
- **Collection endpoints** take query parameters through `GET` and page with `offset` and `next_offset`, or with `cursor` and `next_cursor` for the scanned collections.
- **Path parameters** are written `:name`, for example `/v1/science/spaces/:key`.
- Examples use `$AURIC_API_KEY` for your key.
- **Auth** "None" means the endpoint is public and unmetered. Everything else needs a key; see [Authentication](#).

Pages

Page	Endpoints
Discovery	Index, health, readiness, methods, formats, tiers, status, metrics, catalogue, docs, OpenAPI
Colour and colour science	Convert, contrast, the 63-space registry, dossier, Delta E, colour temperature, illuminants, metrics
Palettes, harmonies, gradients and shades	Generate, evaluate, classify, analyse, export and render
Collections	The colour atlas, nearest colour, and the palette, harmony, gradient and shade libraries
Design tokens and personalisation	Token systems and exports; brief-driven palette search and design systems
Accessibility	WCAG and APCA checks, recommendations, image audits, maps, picks, palette audits, the contrast corpus
Vision and Ishihara	Colour-vision models, conditions, matrices, simulations and audits; Ishihara plates
Analyzer	Stages, markup-only page audits, and rendering reports to HTML or PDF
Usage, jobs, webhooks and GraphQL	Usage history and live feed, async jobs, webhooks, GraphQL

SERVICES > PUBLIC API

Accessibility

Check colour pairs against WCAG 2.2 and APCA, get accessible alternatives, audit images and palettes, map accessible foregrounds on a background, and browse the contrast corpus.

POST /v1/accessibility/check

Check a foreground and background pair against WCAG 2.2 and APCA. 1 credit.

Field	Type	Required	Description
<code>fg</code>	colour	Yes	Text colour. <code>foreground</code> is accepted too.
<code>bg</code>	colour	Yes	The surface behind the text. <code>background</code> is accepted too.
<code>standard</code>	string	No	<code>WCAG 2.1</code> , <code>WCAG 2.2</code> , <code>WCAG 3.0</code> , <code>Auric SD Flexible</code> , <code>Auric SD Strict</code> , or <code>all</code> (the default)

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/check" \  
  -H "x-api-key: $AURIC_API_KEY" \  
  -H "content-type: application/json" \  
  -d '{"fg":"#767676","bg":"#ffffff"}'
```

POST /v1/accessibility/recommend

Suggest the nearest accessible alternative to a failing pair. It keeps the hue and moves the lightness, so the fix still looks like the brand. `target` chooses which colour may move; set the level with `minRatio`.

Field	Type	Required	Description
<code>fg</code>	colour	Yes	Text colour (<code>foreground</code> also accepted)
<code>bg</code>	colour	Yes	Surface colour (<code>background</code> also accepted)
<code>target</code>	string	No	Which side to move: <code>foreground</code> (default) or <code>background</code>
<code>minRatio</code>	number	No	Contrast ratio to reach. Default 4.5.

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/recommend" \  
  -H "x-api-key: $AURIC_API_KEY" \  
  -H "content-type: application/json" \  
  -d '{"fg":"#c9a227","bg":"#ffffff","target":"foreground","minRatio":4.5}'
```

POST /v1/accessibility/from-image

Audit the colour contrast present in an image. 10–12 credits.

Field	Type	Required	Description
<code>image</code>	string	Yes	Base64 or a data URL (up to 4 MB)

POST /v1/accessibility/map

The whole territory of foreground colour a background leaves you. For each hue, it returns the lightness at which the most saturated colour of that hue just reaches the target on this background. `territory_pct` is the share of the slice that qualifies, so one background can be compared with another. A `null` lightness means that hue never reaches the target at any lightness.

Field	Type	Required	Description
<code>ground</code>	colour	Yes	The background colour
<code>targets</code>	number array	No	Contrast ratios to contour. Default <code>[3, 4.5, 7]</code> , at most 6.
<code>resolution</code>	number	No	Hues sampled around the circle, 12–360. Default 72.

Returns `ground`, `polarity` and `contours`.

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/map" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"ground":"#1D2A3E","targets":[4.5,7]}'
```

POST /v1/accessibility/picks

A ready set of foreground colours that clear a target on one background. It walks the hue circle, takes each hue to its boundary lightness, then steps a margin past it so rounding cannot land a pick under the target. Hues that cannot make it are dropped and counted.

Field	Type	Required	Description
<code>ground</code>	colour	Yes	The background colour
<code>target</code>	number	No	Contrast ratio to clear. Default 4.5.
<code>count</code>	number	No	Hues to try, 1–72. Default 12.
<code>margin</code>	number	No	Lightness held back from the boundary, 0–0.5. Default 0.10.

Returns `count`, `dropped` and `picks`.

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/picks" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"ground":"#1D2A3E","target":4.5,"count":12}'
```

POST /v1/accessibility/audit

Audit a palette against one background, with a fix for each failure. The fix keeps the hue and moves the lightness onto the contour. Where a hue cannot reach the target on that background at all, the response says so instead of returning a colour that does not work.

Field	Type	Required	Description
<code>ground</code>	colour	Yes	The background everything sits on
<code>colors</code>	colour array	Yes	The palette, 1–64 colours
<code>target</code>	number	No	Contrast ratio to clear. Default 4.5.

Returns `verdict`, `passing`, `failing` and `results`.

```
curl -X POST "https://api.auricartisan.com/v1/accessibility/audit" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"ground": "#1D2A3E", "colors": ["#D3AF37", "#4A5568", "#F7FAFC"], "target": 4.5}'
```

The contrast corpus

A corpus of five million colour pairs, derived from a seed rather than stored, so paging is cheap and any record can be recomputed. Most pairs are unusable: 88% of the corpus fails AA for body text, so a filter for rare pairs scans many records per result.

GET /v1/accessibility/corpus

Parameter	Description
<code>min_ratio</code> , <code>max_ratio</code>	Contrast ratio bounds, 1–21
<code>level</code>	<code>AA</code> , <code>AAA</code> , <code>fail</code> or <code>any</code> (default)
<code>grade</code>	Auric SD grade: <code>A+</code> , <code>A</code> , <code>B</code> , <code>C</code> , <code>D</code> , <code>F</code>
<code>polarity</code>	<code>BoW</code> (dark on light), <code>WoB</code> or <code>any</code>
<code>limit</code>	Records per page, 1–200. Default 24.
<code>cursor</code>	Corpus index to resume from; use <code>next_cursor</code>
<code>max_scan</code>	Records this call may examine. Default 100,000, maximum 1,000,000.
<code>full</code>	<code>true</code> for the complete record, including all ten vision models

Returns `count`, `scanned`, `hit_rate`, `next_cursor`, `exhausted`, `truncated` and `records`. When `truncated` is `true`, the scan budget ran out before the page filled: follow `next_cursor`.

```
curl "https://api.auricartisan.com/v1/accessibility/corpus?level=AAA&limit=5" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/accessibility/corpus/stats

The measured distribution of contrast across the corpus, from a deterministic sample, so the same sample always gives the same numbers.

Parameter	Description
<code>sample</code>	Records to measure, 1,000–1,000,000. Default 200,000.

Returns `sample`, `mean_contrast_ratio`, `share_at_least`, `share_failing_aa_body` and `distribution`.

GET /v1/accessibility/corpus/:record_id

Resolve one record, for citation: put `acc_2j` in a report and anyone can recover the same two colours.

`:record_id` is a record id such as `acc_2j`, or a corpus index such as `91`. An id that does not parse, or an index outside the corpus, returns `VALIDATION_ERROR`. Returns `record`, `cite` and `recompute`.

```
curl "https://api.auricartisan.com/v1/accessibility/corpus/acc_2j" \
-H "x-api-key: $AURIC_API_KEY"
```

Related pages

- [Vision and Ishihara](#)
- [Colour and colour science](#), for `/v1/color/contrast`
- [Limits and accuracy](#)

SERVICES > PUBLIC API

Analyzer

Audit web pages from their markup through the API — stages and scope, what page source can and cannot answer, why there is no overall score, rendering reports, errors, recipes and costs.

Three endpoints put the URL Analyzer behind a REST call: ask what it can check, run the checks you want against one page, and turn the result into a document. The full guide is at <https://auricartisan.com/library/documentation/api/analyzer/>.

Method	Path	Credits	Purpose
GET	<code>/v1/analyzer/stages</code>	1	What the analyzer can check, and how much of it markup alone can answer
POST	<code>/v1/analyzer/inspect</code>	12	Audit one page from its markup. No browser, no rendering.
POST	<code>/v1/analyzer/render</code>	10	Turn a report into an HTML or PDF document

When to use the API, and when to use the tool

The analyzer on the website and this API differ in one way that decides which you want: the API reads a page's **source**; the tool runs the page in a real browser.

	The tool	This API
Runs the page	Yes, in your browser	No: it reads the source
Contrast, accessibility engine, deep DOM	Yes	Refused, with a reason
SEO, metadata, security headers, sitemap, responsive signals	Yes	Yes
Good for	One page, looked at closely	Many pages, on a schedule
Paid with	Tool tokens	API credits

Use the API to watch many pages for what a crawler can see: a title that went missing after a deploy, a canonical pointing at staging, a weakened Content-Security-Policy, an image that returns 404. Use the tool when you need answers that exist only once a page has been laid out and painted. API credits and tool tokens are separate balances, and neither pays for the other.

Quick start

Audit one page for search-engine metadata:

```
curl -X POST "https://api.auricartisan.com/v1/analyzer/inspect" \  
  -H "x-api-key: $AURIC_API_KEY" \  
  -H "content-type: application/json" \  
  -d '{ "url": "https://example.com/", "scope": ["seo"] }'
```

Ask what else it can check:

```
curl "https://api.auricartisan.com/v1/analyzer/stages" \  
  -H "x-api-key: $AURIC_API_KEY"
```

Turn a result into a PDF:

```
curl -X POST "https://api.auricartisan.com/v1/analyzer/render" \  
  -H "x-api-key: $AURIC_API_KEY" \  
  -H "content-type: application/json" \  
  -d '{"report": $REPORT, "format": "pdf"}' \  
  --output audit.pdf
```

Stages and scope

A full audit has fourteen **stages**. **scope** is the list of stage ids you want, and it is how you ask for less: send **["palette"]** and you get the colours, not an audit with the colours in it.

```
{ "url": "https://example.com/", "scope": ["seo", "security"] }
```

- Omit **scope** to get everything page source can answer.
- Send **[]** and the request is refused (**EMPTY_SCOPE**); an empty list is not treated as "everything".
- **structure**, reading the page at all, is always on. Stages a request depends on are added and echoed back:

```
"scope": {  
  "requested": ["palette"],  
  "effective": ["structure", "palette"],  
  "added_by_dependency": ["structure"]  
}
```

Sections narrow the response, not the work

The response groups the report into **sections**, such as **core**, **seo**, **palette** and **security**. **sections** in the request narrows what comes back; **core** is always included. It does not make a run cheaper: use **scope** to do less, and **sections** to receive less.

```
{ "url": "https://example.com/", "scope": ["seo"], "sections": ["seo"] }
```

What page source can and cannot answer

Every stage carries a **markup** value:

markup	Meaning	Stages
full	Answered completely from the source	structure , project , seo , security , sitemap , responsive
partial	A real but narrower answer, stated in incomplete[]	palette , media
none	Refused with RENDER_REQUIRED ; never returned empty or guessed	contrast , ally , deep , performance , beforeafter , simulation

Contrast needs the colour a browser computed for each element after the whole cascade. Reading colours out of stylesheet text would pair colours that may never appear together on screen, which is a confident, wrong report. The same applies to the accessibility engine (which uses layout and visibility), target sizes (measured boxes), performance (a real page load) and the rendered-versus-raw comparison.

For the **partial** stages:

- **palette** returns every colour the stylesheets declare, which is not the same as the colours covering the most pixels;
- **media** returns every image, video and audio file in the markup, with its real size fetched; anything JavaScript inserts is invisible.

```
"incomplete": [  
  {  
    "stage": "palette",  
    "reason": "Declared colours only. The dominant palette is sampled from rendered pixels, which needs a  
browser."  
  }  
]
```

If **incomplete** is empty, nothing was narrowed.

Why there is no overall score

/v1/analyzer/inspect always returns **score: null**. An overall score averages categories, and a markup-only run does not measure some of them; leaving them out makes the score incomparable, and filling them in makes it partly fiction. Instead you get the categories that were measured and a list of those that were not:

```
"score": null,  
"category_scores": { "metadata": 84, "seo": 85, "a11y": 68 },  
"unscored_categories": [  
  "accessibility", "a11yPlus", "contrast", "performance",  
  "reliability", "security", "media", "visual", "deep", "responsive"  
],  
"scoreCoverage": { "measured": 3, "of": 13, "unmeasured": ["accessibility", "..."] }
```

If you want one number, compute it from **category_scores** with weights you choose. For a comparable overall score, run the audit in the tool.

Note: The **a11y** category is the markup-level accessibility count: heading order, alt text, form labels, landmarks and **lang**. It is not the analyzer's 112-rule accessibility engine, which is **accessibility** and **a11yPlus**, always unscored here.

GET /v1/analyzer/stages

The vocabulary, generated from the analyzer's own stage table. Call it once, cache it, and build scopes from it. 1 credit, no body.

Returns `checks_per_token`, `always_on`, `stage_ids`, `stages`, `sections` and `notes`. Each stage has `id`, `name`, `does`, `section`, `needs_render`, `markup`, `endpoint`, `caveat` and `why_unavailable`. Filtering on `markup` not equal to `"none"` gives every stage `/inspect` accepts.

```
{
  "id": "seo",
  "name": "SEO and metadata",
  "does": "Title, description, headings, schema, robots, canonical, keyword density.",
  "section": "seo",
  "needs_render": false,
  "markup": "full",
  "endpoint": "/v1/analyzer/inspect",
  "caveat": null,
  "why_unavailable": null
}
```

POST /v1/analyzer/inspect

12 credits, one page per call.

Field	Type	Required	Description
<code>url</code>	string	Yes	Absolute <code>http</code> or <code>https</code> . Private and reserved addresses are refused.
<code>scope</code>	string array	No	Stage ids. Omit for everything markup can answer; <code>[]</code> is refused.
<code>sections</code>	string array	No	Narrows the response only; <code>core</code> is always included

The response includes `url`, `final_url`, `audit_version`, `scope`, `score` (always `null`), `unscored_categories`, `category_scores`, `timing`, `section_index`, `sections` and `incomplete`. `section_index` summarises each section's findings count and size, so you can log it without keeping the whole report.

Only what was measured comes back. A stage outside your scope contributes nothing: asking for `["seo"]` returns no security section, and no run returns contrast pairs, the accessibility engine's audit or the deep analysis.

POST /v1/analyzer/render

10 credits. Takes a report, one `/inspect` returned or one you stored, and renders it. No page is fetched, so it works on an old report.

Field	Type	Required	Description
<code>report</code>	object	Yes	An analyzer report
<code>format</code>	string	No	<code>html</code> (default) or <code>pdf</code>

format	Content type	Notes
html	text/html	A self-contained document, around 14 KB for a small page
pdf	application/pdf	Laid out on the server, around 7 KB for the same page. Text and rules only, no screenshots.

The PDF uses the standard PDF fonts, which cover Latin characters; content in other scripts will not appear. Use `html` for pages not in a Latin script. Markdown reports are not offered through the API; the tool still writes them from a saved audit.

Errors

These are decided before your page is fetched, so a rejected request does no work.

Code	Status	When
UNKNOWN_STAGE	400	A stage id that does not exist; the response lists the fourteen real ones
EMPTY_SCOPE	400	<code>scope: []</code> . Omit the field for everything, or name what you want.
RENDER_REQUIRED	400	A stage that needs a browser; names the offenders and what is available
INVALID_INPUT	400	A missing or unusable <code>url</code>
UNKNOWN_FORMAT	400	<code>/render</code> with a format other than <code>html</code> or <code>pdf</code>
FETCH_FAILED	502	Your page could not be read: DNS, TLS, a timeout, or a private address
ANALYZER_UNAVAILABLE	503	The analyzer backend is unavailable; <code>/v1/analyzer/stages</code> still works

```
{
  "error": {
    "code": "RENDER_REQUIRED",
    "message": "contrast needs a rendered page, which this endpoint does not do. GET /v1/analyzer/stages lists what markup alone can answer.",
    "details": {
      "needs_render": ["contrast"],
      "available": ["structure", "project", "palette", "seo", "media", "responsive", "security", "sitemap"]
    }
  },
  "request_id": "7140da9b-0ddf-417d-9c8b-9775a5056a01"
}
```

The shared 401, 403, 404, 405, 413 and 429 responses are on [Errors](#).

Recipes

Watch a page for metadata regressions

```
const res = await fetch("https://api.auricartisan.com/v1/analyzer/inspect", {
  method: "POST",
  headers: { "x-api-key": process.env.AURIC_API_KEY, "content-type": "application/json" },
  body: JSON.stringify({ url, scope: ["seo"], sections: ["seo"] }),
});
const { sections, category_scores } = await res.json();
const meta = sections.seo.metadataAudit;
if (category_scores.seo < 80) {
  console.error(`${url}: SEO fell to ${category_scores.seo}`);
  for (const finding of sections.seo.seoAudit.findings) console.error(` ${finding.title}`);
  process.exitCode = 1;
}
```

Audit every page in a sitemap

One call per page, paced against your rate limit: 300 requests a minute on Specialist, 3,000 on Industrial Pro.

```
for (const url of urls) {
  const res = await fetch(endpoint, { method: "POST", headers, body: JSON.stringify({ url, scope: ["seo",
"security"] }) });
  if (res.status === 429) { // slow down and retry this one
    await new Promise((r) => setTimeout(r, Number(res.headers.get("retry-after") || 5) * 1000));
    continue;
  }
  const audit = await res.json();
  results.push({ url, scores: audit.category_scores, findings: audit.sections });
}
```

`x-quota-remaining` comes back on every response, so you can stop before you run out.

Just the colours

```
curl -X POST "https://api.auricartisan.com/v1/analyzer/inspect" \
-H "x-api-key: $AURIC_API_KEY" -H "content-type: application/json" \
-d '{ "url": "https://example.com/", "scope": ["palette"], "sections": ["palette"] }'
```

This returns `sections.palette.colors` (the declared colours, grouped by where they are used) and `sections.palette.colorAudit`, with the "declared, not sampled" caveat in `incomplete`.

A PDF for a client, from a stored audit

```
const audit = await inspect(url, ["seo", "security", "responsive"]);
const report = Object.assign({}, ...Object.values(audit.sections));
const pdf = await fetch("https://api.auricartisan.com/v1/analyzer/render", {
  method: "POST",
  headers: { "x-api-key": key, "content-type": "application/json" },
  body: JSON.stringify({ report, format: "pdf" }),
});
await writeFile("audit.pdf", Buffer.from(await pdf.arrayBuffer()));
```

`/render` takes the whole report, so merge the sections back together before sending it.

What is not here yet

- **A rendered scan.** Contrast, the accessibility engine and the deep DOM comparison need a real browser. Until the API is connected to one, those stages are refused here and available in the tool.
- **A crawl.** Auditing a whole site takes minutes, which needs a job you start and poll. Until then, fetch a sitemap yourself and call `/inspect` per page.

Credits

The price is per call, not per stage. A scope of one stage and a scope of eight cost the same 12 credits. Narrowing the scope makes a run faster and the response smaller, not cheaper; if you want the whole picture, ask for it in one call.

Related pages

- [Plans, credits and quotas](#)
- The website's Analyzer tool: [Analyzer docs](#)

SERVICES > PUBLIC API

Collections

Browse the colour atlas, find the nearest catalogued colour, and filter the palette, harmony, gradient and shade-scale collections.

The API exposes large, measured collections you can filter and cite. Two kinds exist:

- **Finite catalogues** of 8,192 records (the colour atlas and the harmony, gradient and shade libraries). A filtered browse reports `matched`, a real total, and pages with `offset` and `next_offset`.
- **Scanned collections** too large to count in one request (five million palettes, and the five-million-pair accessibility corpus described in [Accessibility](#)). A filtered browse scans a budget of records and reports `scanned`, `hit_rate`, `next_cursor`, `exhausted` and `truncated`.

Important: In a scanned collection, `truncated: true` means the scan budget ran out before the page filled, not that nothing matches. Follow `next_cursor`, or raise `max_scan`, before concluding a filter is empty. `exhausted: true` means the collection really ended.

Records in the libraries are derived from a published seed and their index rather than stored, so an id such as `pal_1z` or `har_5k` is citable: anyone can recompute the same record.

Colour atlas

GET /v1/color/atlas

Browse the 8,192-colour atlas under a filter. Hue ranges wrap, so `hue_from=340&hue_to=20` gives the reds.

Parameter	Description
<code>hue_from</code> , <code>hue_to</code>	Hue range, 0–360; wraps past 360
<code>min_lightness</code> , <code>max_lightness</code>	OKLCH lightness bounds, 0–1
<code>min_chroma</code>	Chroma floor, 0–1
<code>min_contrast_on_white</code> , <code>min_contrast_on_black</code>	Keep only colours readable on white or black at this ratio
<code>emotion</code> , <code>art_movement</code> , <code>ai_mood</code> , <code>ai_design_usage</code> , <code>design_tag</code>	Label filters, for example <code>calm</code> , <code>Fauvism</code> , <code>serene</code> , <code>data-viz</code> , <code>pastel</code> ; see the stats
<code>sort</code>	<code>id</code> (default), <code>hue</code> , <code>lightness</code> , <code>chroma</code> or <code>contrast_on_white</code>
<code>order</code>	<code>asc</code> (default) or <code>desc</code>
<code>limit</code>	Colours per page, 1–200. Default 24.
<code>offset</code>	Where to start; use <code>next_offset</code>
<code>full</code>	<code>true</code> adds the vision simulations and labels

Returns `matched`, `share_of_atlas`, `next_offset` and `colors`.

```
curl "https://api.auricartisan.com/v1/color/atlas?min_contrast_on_white=4.5&sort=chroma&order=desc&limit=5" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/color/atlas/stats

What the atlas contains: hue occupancy, the lightness spread, the label vocabulary, and how much of it works as text on white or black. Returns `readable_as_text`, `hue_distribution`, `lightness_distribution` and `labels`.

GET /v1/color/atlas/:id

One sample, whole. `:id` is an id from 0 to 8191, or a hex the atlas contains (URL-encoded, such as `%2376cdf6`). An unknown id or hex returns `VALIDATION_ERROR`. Returns `color`.

POST /v1/color/nearest

The catalogued colours closest to one you have. Distance is CIE76 in Lab, and the response names the metric.

Field	Type	Required	Description
<code>color</code>	colour	Yes	The colour to match
<code>count</code>	number	No	How many matches, 1–50. Default 5.
<code>full</code>	boolean	No	<code>true</code> for the complete record on each match

Returns `metric` and `matches`.

```
curl -X POST "https://api.auricartisan.com/v1/color/nearest" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"color": "#D3AF37", "count": 5}'
```

Palette library (five million palettes)

GET /v1/palette/library

Parameter	Description
<code>method</code>	<code>complementary</code> , <code>triadic</code> , <code>analogous</code> , <code>tetradic</code> , <code>golden_ratio</code> , <code>monochromatic</code> or <code>random</code>
<code>character</code>	Measured character: <code>neutral</code> , <code>monochrome</code> , <code>analogous</code> , <code>triadic</code> or <code>diverse</code>
<code>hue</code>	First colour's hue band, red through pink
<code>min_contrast</code> , <code>max_contrast</code>	Bounds on the worst pair's contrast, 1–21
<code>min_lightness</code> , <code>max_lightness</code>	Mean OKLCH lightness bounds, 0–1
<code>min_chroma</code>	Mean OKLCH chroma floor, 0–0.5
<code>carries_body_text</code>	<code>true</code> for palettes with a pair at 4.5:1 or better
<code>sort</code>	<code>id</code> (default), <code>contrast</code> , <code>spread</code> , <code>lightness</code> or <code>chroma</code> ; other than <code>id</code> , sorting is within the scan
<code>order</code>	<code>asc</code> (default) or <code>desc</code>
<code>limit</code>	Per page, 1–100. Default 24.

Parameter	Description
<code>cursor</code>	Where to resume; use <code>next_cursor</code>
<code>max_scan</code>	Scan budget, 1,000–500,000. Default 20,000. Raise it for a rare filter.
<code>full</code>	<code>true</code> for per-colour RGB and OKLCH and the palette metrics

Returns `scanned`, `hit_rate`, `next_cursor`, `exhausted`, `truncated` and `palettes`.

```
curl "https://api.auricartisan.com/v1/palette/library?carries_body_text=true&limit=5" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/palette/library/stats

Measured on a stated sample (70 runs of 600 consecutive palettes). Returns `measured_on`, `with_an_aa_pair`, `method` and `character`. About 55% of the palettes contain a pair that carries body text.

GET /v1/palette/library/:id

One palette, whole, with the seed and batch that recompute it. `:id` is a palette id (`pal_1z`) or a number from 0 to 4999999.

Harmony library (8,192 harmonies)

GET /v1/harmony/library

Parameter	Description
<code>method</code>	One of the 23 methods, for example <code>triadic</code> ; see the stats
<code>family</code>	<code>complementary</code> , <code>analogous</code> , <code>polyadic</code> , <code>monochromatic</code> or <code>compound</code>
<code>hue</code>	Base hue band: red, orange, yellow, green, cyan, blue, purple, pink
<code>colors</code>	How many members, 2–7
<code>min_contrast</code>	The worst pair must clear this ratio, 1–21
<code>min_adherence</code>	How closely it holds its canonical angles, 0–1
<code>sort</code>	<code>id</code> (default), <code>contrast</code> , <code>spread</code> , <code>adherence</code> or <code>colors</code>
<code>order</code> , <code>limit</code> , <code>offset</code> , <code>full</code>	As in the other catalogues; <code>limit</code> 1–100, default 24

Returns `matched`, `share_of_collection`, `next_offset` and `harmonies`. Only about a quarter of the collection contains a pair that carries body text, so `min_contrast` is the filter worth using.

GET /v1/harmony/library/stats

Returns `with_an_aa_pair`, `method`, `family` and `color_count`.

GET /v1/harmony/library/:id

One harmony, whole, with its derivation. `:id` is a harmony id (`har_5k`) or a number from 0 to 8191.

Gradient library (8,192 gradients)

GET /v1/gradient/library

Parameter	Description
<code>scheme</code>	For example <code>Aurora</code> , <code>Spectral</code> , <code>Terrain</code> ; see the stats
<code>complexity</code>	<code>simple</code> , <code>detailed</code> or <code>extreme</code>
<code>banding</code>	<code>smooth</code> , <code>subtle-compression</code> or <code>visible-step-risk</code>
<code>type</code>	<code>linear</code> , <code>radial</code> or <code>conic</code>
<code>min_score</code>	Quality floor, 0–1
<code>min_stops</code> , <code>max_stops</code>	Bounds on the number of colour stops
<code>sort</code>	<code>id</code> (default), <code>score</code> or <code>stops</code>
<code>order</code> , <code>limit</code> , <code>offset</code> , <code>full</code>	<code>limit</code> 1–100, default 24; <code>full</code> adds interpolation space, easing and metrics

Returns `matched` , `share_of_collection` , `next_offset` and `gradients` . About one gradient in five steps visibly somewhere; `banding` is the filter worth using.

GET /v1/gradient/library/stats

Returns the `banding` , `scheme` and `interpolation_space` tallies.

GET /v1/gradient/library/:id

One gradient, whole. `:id` is a row number from 0 to 8191.

Shade library (8,192 scales)

GET /v1/shade/library

Parameter	Description
<code>method</code>	One of ten, for example <code>tailwind-like</code> , <code>oklch-ramp</code> , <code>ink-paper</code>
<code>evenness</code>	<code>even</code> , <code>slightly-uneven</code> or <code>lumpy</code>
<code>hue</code>	Base hue band, red through pink
<code>steps</code>	Exactly this many steps, 2–64

Parameter	Description
<code>min_span</code>	Contrast between the extremes, 1–21
<code>min_score</code>	Quality floor, 0–1
<code>monotonic</code>	<code>true</code> for scales that never turn round
<code>carries_body_text</code>	<code>true</code> for scales with a pair at 4.5:1 or better
<code>sort</code>	<code>id</code> (default), <code>score</code> , <code>steps</code> , <code>span</code> or <code>evenness</code>
<code>order</code> , <code>limit</code> , <code>offset</code> , <code>full</code>	<code>limit</code> 1–100, default 24

Returns `matched`, `share_of_collection`, `next_offset` and `scales`. Every scale is monotonic and can carry text; evenness is what separates them, and only 8.2% are clean.

GET /v1/shade/library/stats

Returns `evenness`, `monotonic`, `carries_body_text` and `method`.

GET /v1/shade/library/:id

One scale with every measurement. `:id` is a shade id (`shade_000000`) or a row number from 0 to 8191.

Related pages

- [Accessibility](#), for the contrast corpus
- [Palettes, harmonies, gradients and shades](#)

SERVICES > PUBLIC API

Colour and colour science

Convert colours, check a contrast ratio, and use the 63-space colour registry, dossiers, Delta E, colour temperature, illuminants and difference metrics.

POST /v1/color/convert

Convert one colour between the common spaces. 1 credit. For the full 63-space registry with white-point control, use `/v1/science/convert`.

Field	Type	Required	Description
<code>color</code>	colour	Yes	<code>#rrggbb</code> , <code>#rgb</code> , <code>rgb(r,g,b)</code> , <code>[r,g,b]</code> or <code>{r,g,b}</code>
<code>to</code>	string	No	One target space. Omit it to get every common space at once.

```
curl -X POST "https://api.auricartisan.com/v1/color/convert" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"color":"#c9a227","to":"lab"}'
```

POST /v1/color/contrast

The WCAG contrast ratio between two colours, with pass or fail at each level. 1 credit.

Field	Type	Required	Description
<code>fg</code>	colour	Yes	Foreground colour
<code>bg</code>	colour	Yes	Background colour
<code>deltaE</code>	number	No	Difference formula: 76, 94 or 2000 (the default)

Returns `ratio` (number), `aa`, `aaa` and `aa_large` (booleans).

```
curl -X POST "https://api.auricartisan.com/v1/color/contrast" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"fg":"#767676","bg":"#ffffff"}'
```

For WCAG 2.2 and APCA together, use `/v1/accessibility/check`.

GET /v1/science/spaces

Every colour space the platform knows, with provenance: sixty-three spaces across RGB working spaces, CIE spaces, perceptual and appearance models, HDR containers, print and video.

Parameter	In	Required	Description
<code>category</code>	query	No	<code>rgb</code> , <code>cie</code> , <code>perceptual</code> , <code>cylindrical</code> , <code>appearance</code> , <code>hdr</code> , <code>print</code> , <code>video</code> , <code>order</code> or <code>opponent</code>
<code>full</code>	query	No	<code>true</code> to include matrices, primaries and transfer functions

```
curl "https://api.auricartisan.com/v1/science/spaces?category=perceptual" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/science/spaces/:key

One colour space in full, including its conversion matrices.

Parameter	In	Required	Description
key	path	Yes	For example <code>oklch</code> , <code>display-p3</code> or <code>acescct</code>

An unknown key returns `404 UNKNOWN_SPACE`, and the response suggests near matches.

```
curl "https://api.auricartisan.com/v1/science/spaces/oklch" \  
-H "x-api-key: $AURIC_API_KEY"
```

POST /v1/science/convert

Convert a colour into any registry space, or several at once. 1 credit.

Field	Type	Required	Description
color	colour	Yes	The colour to convert
to	string or string array	Yes	One space key, or an array of them
white_point	string	No	Reference white; default D65

```
curl -X POST "https://api.auricartisan.com/v1/science/convert" \  
-H "x-api-key: $AURIC_API_KEY" \  
-H "content-type: application/json" \  
-d '{"color":"#c9a227","to":["lab","oklch","display-p3"]}'
```

POST /v1/science/dossier

One colour in every space at once, plus its colour temperature. 5 credits. It is expensive compared with convert: use it when you want everything, not three spaces.

Field	Type	Required	Description
color	colour	Yes	The colour
white_point	string	No	Reference white; default D65
category	string	No	Restrict to one category

```
curl -X POST "https://api.auricartisan.com/v1/science/dossier" \  
-H "x-api-key: $AURIC_API_KEY" \  
-H "content-type: application/json" \  
-d '{"color":"#c9a227}"
```

POST /v1/science/delta-e

The perceptual difference between two colours, 2–4 credits. It returns CIE76, CIE94, CIEDE2000 and CMC together, with a plain-language band. CIEDE2000 is the headline figure and is verified against the Sharma reference set.

Field	Type	Required	Description
<code>from</code>	colour	Yes	First colour
<code>to</code>	colour	Yes	Second colour
<code>metric</code>	string	No	<code>cie76</code> (the default headline), <code>cie94</code> , <code>cie76</code> , <code>cmc</code> or <code>all</code>
<code>application</code>	string	No	CIE94 only: <code>graphic</code> (default) or <code>textile</code>
<code>l</code>	number	No	CMC lightness weight; default 2
<code>c</code>	number	No	CMC chroma weight; default 1

Returns `delta_e` (number), `band` (string) and `metrics` (object).

```
curl -X POST "https://api.auricartisan.com/v1/science/delta-e" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"from":"#c9a227","to":"#d3af37}"'
```

POST /v1/science/temperature

The correlated colour temperature (CCT) of a colour, or the colour of a temperature. 2–4 credits. Send `color` to measure or `kelvin` to render. It always returns Duv alongside the CCT: beyond about ± 0.05 the colour is too far from the Planckian locus for a temperature to describe it, and `meaningful` says so.

Field	Type	Required	Description
<code>color</code>	colour	No	Measure this colour; not with <code>kelvin</code>
<code>kelvin</code>	number	No	Render this temperature, 1667–25000 K

Returns `cct_kelvin` (number), `duv` (number), `meaningful` (boolean) and `tint` (string).

```
curl -X POST "https://api.auricartisan.com/v1/science/temperature" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"color":"#fff5e1}"'
```

GET /v1/science/illuminants

Standard reference illuminants with their chromaticities.

```
curl "https://api.auricartisan.com/v1/science/illuminants" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/science/metrics

The difference formulae, their options, and when to use each.

```
curl "https://api.auricartisan.com/v1/science/metrics" \
-H "x-api-key: $AURIC_API_KEY"
```

Related pages

- [Collections](#), including the colour atlas and nearest-colour search
- [Glossary](#)

SERVICES > PUBLIC API

Discovery endpoints

The public endpoints that describe the API itself — index, health, readiness, methods, formats, tiers, status, metrics, catalogue and documentation.

These endpoints describe the API. All but one are public and unmetered, so you can evaluate the API before buying.

Method	Path	Auth	Purpose
GET	/v1	None	Index of every endpoint, with the current version
GET	/v1/health	None	Liveness probe; always cheap, never touches storage
GET	/v1/ready	None	Readiness probe: whether each backing store is bound
GET	/v1/methods/:tool	Key	The algorithms a given tool supports
GET	/v1/formats	None	Every export format, across every exportable resource
GET	/v1/tiers	None	Rate limit and monthly quota for each tier
GET	/v1/metrics	None	Process-level counters in Prometheus format
GET	/v1/status	None	Human-readable status page
GET	/v1/catalog	None	Every endpoint with its parameters and examples, as JSON

Method	Path	Auth	Purpose
GET	<code>/v1/docs</code>	None	Interactive reference (Redoc)
GET	<code>/v1/openapi.yaml</code>	None	The OpenAPI 3.1 document

GET /v1

Returns `version` (string) and `endpoints` (string array).

```
curl "https://api.auricartisan.com/v1"
```

GET /v1/health

Returns `ok` (boolean) and `time` (ISO 8601).

```
curl "https://api.auricartisan.com/v1/health"
```

GET /v1/ready

Returns `ready` (boolean) and `checks` (object). Use it in a deployment gate: `/v1/health` says the service is up, while `/v1/ready` says it can actually serve.

```
curl "https://api.auricartisan.com/v1/ready"
```

GET /v1/methods/:tool

The algorithms a tool supports. Use it to find valid `method` values, for example for `/v1/palette/generate` or `/v1/harmony/generate`.

Parameter	In	Required	Description
<code>tool</code>	path	Yes	For example <code>palette</code> , <code>harmony</code> or <code>vision</code>

```
curl "https://api.auricartisan.com/v1/methods/harmony" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/formats

Every export format for every exportable resource. The export endpoints refer to it for valid `format` values.

```
curl "https://api.auricartisan.com/v1/formats"
```

GET /v1/tiers

The machine-readable plan table: rate limit and monthly quota for each tier. See [Plans, credits and quotas](#).

```
curl "https://api.auricartisan.com/v1/tiers"
```

GET /v1/catalog

The catalogue as JSON: every endpoint with its parameters and examples. It is generated from the same source as the OpenAPI document, so a client can build its own reference or check its coverage.

```
curl "https://api.auricartisan.com/v1/catalog"
```

GET /v1/docs and GET /v1/openapi.yaml

[/v1/docs](#) is the interactive reference in the browser. [/v1/openapi.yaml](#) is the OpenAPI 3.1 document, for generating clients or importing into API tools.

```
curl "https://api.auricartisan.com/v1/openapi.yaml"
```

GET /v1/status and GET /v1/metrics

[/v1/status](#) is a human-readable status page. [/v1/metrics](#) exposes process-level counters in Prometheus format.

Related pages

- [Reference](#)
- [Getting started](#)

SERVICES > PUBLIC API

Palettes, harmonies, gradients and shades

Generate, score, classify, analyse, export and render palettes, harmonies, gradients and shade scales.

These endpoints build colour sets and measure the ones you already have. Valid **method** values come from [GET /v1/methods/:tool](#), and valid export **format** values from [GET /v1/formats](#).

Palettes

POST /v1/palette/generate

Generate a palette from a seed colour. 2–4 credits.

Field	Type	Required	Description
<code>method</code>	string	No	See <code>/v1/methods/palette</code> . Default <code>triadic</code> .
<code>hue</code>	number	No	Base hue, 0–360. Random when omitted.
<code>saturation</code>	number	No	0–1. Default 0.7.
<code>lightness</code>	number	No	0–1. Default 0.55.
<code>count</code>	integer	No	How many colours, 1–64. Default 5.
<code>seed</code>	integer	No	For a repeatable palette

```
curl -X POST "https://api.auricartisan.com/v1/palette/generate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"method":"triadic","hue":45,"saturation":0.7,"lightness":0.55,"count":5}'
```

POST /v1/palette/evaluate

Score a palette for harmony, contrast and accessibility.

Field	Type	Required	Description
<code>palette</code>	object	Yes	<code>{ colors: [{ rgb: [r,g,b] }, ...] }</code> , the shape <code>/v1/palette/generate</code> returns

```
curl -X POST "https://api.auricartisan.com/v1/palette/evaluate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"palette":{"colors":[{"rgb":[201,162,39]},{"rgb":[29,42,62]},{"rgb":[255,252,247]}]}}'
```

POST /v1/palette/export

Render a palette into a file format.

Field	Type	Required	Description
<code>format</code>	string	Yes	<code>hex</code> , <code>json</code> , <code>css</code> , <code>scss</code> , <code>tailwind</code> , <code>ase-json</code> or <code>svg</code> ; see <code>/v1/formats</code>
<code>data</code>	object	Yes	<code>{ colors: [{ rgb: [r,g,b] }, ...] }</code>
<code>options</code>	object	No	Format-specific options

```
curl -X POST "https://api.auricartisan.com/v1/palette/export" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"format":"css","data":{"colors":[{"rgb":[201,162,39]},{"rgb":[29,42,62]}]}}'
```

POST /v1/palette/from-image

Extract a palette from an uploaded image. 10–12 credits.

Field	Type	Required	Description
<code>image</code>	string	Yes	Base64 or a data URL. Image bodies are limited to 4 MB.
<code>count</code>	integer	No	Colours to extract

POST /v1/palette/png

Render a palette as a PNG swatch sheet. 10–12 credits. Returns `image/png`.

Field	Type	Required	Description
<code>colors</code>	colour array	Yes	The palette to draw

Harmonies

POST /v1/harmony/generate

Build a harmony set (complementary, triadic, analogous and others) from a base colour. 2–4 credits.

Field	Type	Required	Description
<code>method</code>	string	Yes	See <code>/v1/methods/harmony</code>
<code>hue</code>	number	Yes	Base hue, 0–360
<code>saturation</code>	number	Yes	0–1
<code>lightness</code>	number	Yes	0–1

```
curl -X POST "https://api.auricartisan.com/v1/harmony/generate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"method":"triadic","hue":45,"saturation":0.7,"lightness":0.55}'
```

POST /v1/harmony/classify

Identify which harmony an existing set of colours follows.

Field	Type	Required	Description
<code>colors</code>	colour array	Yes	The colours to classify

POST /v1/harmony/identify

Name the scheme your colours form. It takes 2 to 12 colours and matches their hue pattern against all 23 schemes. Each colour is tried as the anchor in turn, so the order you send them in does not matter. Schemes with the same shape on the hue circle that differ only in lightness or saturation come back together in `ties`.

Field	Type	Required	Description
<code>colors</code>	array	Yes	2 to 12 colours as <code>["#rrggbb", ...]</code>

Returns `verdict`, `best_match`, `ties`, `candidates`, `hue_offsets`, `min_pair_contrast` and `carries_body_text`.

```
curl -X POST "https://api.auricartisan.com/v1/harmony/identify" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"colors":["#FF0000","#00FF00","#0000FF"]}'
```

POST /v1/harmony/export

Export a harmony set.

Field	Type	Required	Description
<code>colors</code>	colour array	Yes	The harmony set
<code>format</code>	string	Yes	See <code>/v1/formats</code>

Gradients

POST /v1/gradient/generate

Interpolate a gradient between stops, in a chosen space. 2–4 credits.

Field	Type	Required	Description
<code>stops</code>	colour array	Yes	Two or more colours
<code>steps</code>	integer	No	Samples to return
<code>space</code>	string	No	Interpolation space; <code>oklab</code> avoids the grey dip that <code>srgb</code> produces

```
curl -X POST "https://api.auricartisan.com/v1/gradient/generate" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"stops":["#c9a227","#1d2a3e"],"steps":7,"space":"oklab"}'
```

POST /v1/gradient/analyse

Measure a gradient you already have: will it band, and where? It samples the ramp, converts each sample to OKLab and measures the perceptual distance between neighbouring samples. The verdict compares the worst step with the average one, so it does not change when you ask for more samples. Measured across 3,000 labelled gradients, that ratio has a median of 1.55 for smooth gradients and 5.09 for step-risk ones.

Field	Type	Required	Description
<code>stops</code>	array	Yes	2 to 64 stops, as <code>["#rrggbb", ...]</code> (evenly spaced) or <code>[{pos, hex}, ...]</code> with <code>pos</code> 0–1
<code>steps</code>	number	No	Samples along the ramp, 8–1024. Default 96.

Returns `banding`, `step_ratio`, `delta_e_mean`, `delta_e_max`, `worst_transition_at` (where the worst step is) and `note`.

```
curl -X POST "https://api.auricartisan.com/v1/gradient/analyse" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"stops":["#001a33","#003d66","#ffcc00"]}'
```

POST /v1/gradient/export

Export a gradient as CSS, SVG or a design-tool format.

Field	Type	Required	Description
<code>stops</code>	colour array	Yes	The gradient stops
<code>format</code>	string	Yes	See <code>/v1/formats</code>

POST /v1/gradient/png

Render a gradient as a PNG. 10–12 credits. Returns `image/png`.

Field	Type	Required	Description
<code>stops</code>	colour array	Yes	The gradient stops

Shades

POST /v1/shade/scale

Build a tint and shade scale from one colour. 2–4 credits.

Field	Type	Required	Description
<code>color</code>	colour	Yes	The base colour
<code>steps</code>	integer	No	Steps in the scale

```
curl -X POST "https://api.auricartisan.com/v1/shade/scale" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"color":"#c9a227","steps":9}'
```

POST /v1/shade/evaluate

Measure a scale you already have: does it work as a token ramp? A scale fails in one of three ways, each with a different fix, and the response says which: it **turns round** (two steps read as the same tone), its steps are **lumpy** (the largest is several times the average), or its extremes are **too close** to set text between.

Field	Type	Required	Description
<code>steps</code>	array	Yes	3 to 64 colours as <code>["#rrggbb", ...]</code> , in scale order

Returns `verdict`, `notes`, `monotonic`, `step_ratio`, `evenness`, `usable_span` and `text_pairs`.

```
curl -X POST "https://api.auricartisan.com/v1/shade/evaluate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"steps":["#FFFFFF","#E0E0E0","#A0A0A0","#606060","#202020"]}'
```

POST /v1/shade/export

Export a shade scale.

Field	Type	Required	Description
<code>color</code>	colour	Yes	Base colour of the scale
<code>format</code>	string	Yes	See <code>/v1/formats</code>

Related pages

- [Collections](#), to browse ready-made palettes, harmonies, gradients and scales

- Design tokens and personalisation

SERVICES > PUBLIC API

Design tokens and personalisation

Generate a contrast-checked design-token system from one colour, export it for ten platforms, and run the brief-driven palette search and design-system generator.

Design tokens

POST /v1/tokens/generate

Turn one seed colour into a complete, contrast-checked token system. 2–4 credits. Ramps are stepped in OKLCH so every hue is perceptually even, chroma follows a gamut-aware envelope so the light and dark ends stay distinct, and foreground roles are measured: `--color-on-primary` is whichever of white or ink actually clears 4.5:1.

Field	Type	Required	Description
<code>seed</code>	colour	Yes	The seed colour
<code>name</code>	string	No	System name used in file headers
<code>harmony</code>	string	No	<code>analogous</code> (default), <code>complementary</code> , <code>triadic</code> or <code>mono</code>
<code>include</code>	string array	No	Any of <code>color</code> , <code>space</code> , <code>radius</code> , <code>type</code> . Default all.
<code>dark</code>	boolean	No	Include dark-mode roles. Default <code>true</code> .

Returns `meta`, `contrast`, `out_of_gamut` and `tokens`.

```
curl -X POST "https://api.auricartisan.com/v1/tokens/generate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"seed":"#c9a227","name":"Auric","harmony":"analogous"}'
```

POST /v1/tokens/export

Render a token deck for a platform. There are ten targets, from CSS custom properties to Jetpack Compose.

Field	Type	Required	Description
<code>tokens</code>	array	Yes	Objects of <code>{ name, value, group? }</code> : the <code>tokens</code> array from generate, or your own
<code>format</code>	string	Yes	See <code>/v1/tokens/formats</code>
<code>name</code>	string	No	System name for the file header
<code>raw</code>	boolean	No	<code>true</code> returns the file itself, as a download, instead of a JSON envelope

```
curl -X POST "https://api.auricartisan.com/v1/tokens/export" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"format": "css", "tokens": [{"name": "--color-primary", "value": "#c9a227"}]}'
```

GET /v1/tokens/formats

The ten token export targets.

Personalisation

These endpoints drive the palette search behind the website's personalisation generator. A **brief** combines seed colours, style keys, an intent and weights.

GET /v1/personalize/vocabulary

Everything a brief may name, with the ranges the engine really applies: the 18 style profiles with their chroma, lightness and saturation windows, the five intents, the 19 hue relationships and their offsets, the 17 type systems, and the gradient and poster catalogues. Read it before writing a brief. Returns **styles**, **intents**, **relationships**, **type_systems** and **limits**.

GET /v1/personalize/presets

The 17 briefs the tool ships, in request shape; each can be posted straight to the endpoints below. Returns **count** and **presets**.

POST /v1/personalize/pool

Grow a working set of colours from seeds and a brief. The search works within this pool. The response also returns the brief resolved to numbers.

Field	Type	Required	Description
colors	string array	No	Seed colours, up to 64. Defaults to the tool's own eight.
styles	string array	No	Style keys from the vocabulary
intent	string	No	ui (default), brand , poster , editorial or dashboard
target	integer	No	Pool size, 1–240. Default 64.
warmth	number	No	0–100. Default 52.
seed	integer	No	Omit for a random one; it is always returned

Returns **brief**, **pool** and **profile**.

```
curl -X POST "https://api.auricartisan.com/v1/personalize/pool" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"colors":["#0F172A","#D3AF37"],"styles":["lux","editorial"],"target":24}'
```

POST /v1/personalize/score

Rank a palette you already have on the search's six scores: preference, harmony, diversity, contrast, style fit and accessibility. Omit **relationship** and every hue geometry is tried, with the best fit reported.

Field	Type	Required	Description
palette	string array	Yes	2–12 colours
relationship	string	No	A relationship id to measure against
styles	string array	No	The brief it is judged for
intent	string	No	One of the five intents
accessibility	number	No	0–100, how hard accessibility is weighted. Default 72.

Returns **score**, **measured_as**, **best_pair** and **alternatives**.

POST /v1/personalize/palettes

Search: N candidates in, the best K out, each with its breakdown. The response reports what the sweep did: how many candidates were sampled, how many survived de-duplication and where the cut fell. Contrast and colour vision are weights in the score, not filters, so a candidate that reads badly is ranked down rather than removed.

Field	Type	Required	Description
colors , styles , intent	—	No	The brief
note	string	No	Free text, matched against curated colour memories
depth	integer	No	Candidates to sample, 8–1200. Default 320.
keep	integer	No	How many to return, 1–120. Default 24.
palette_size	integer	No	Colours per palette, 5–12. Default 6.
diversity	number	No	0–100. Default 68.
accessibility	number	No	0–100. Default 72.
cvd_safe	boolean	No	Weight colour-vision separation. Default true .
unique	boolean	No	Drop repeated signatures. Default true .
seed	integer	No	The same seed and brief always give the same result

Returns **brief**, **sweep** and **palettes**.

```
curl -X POST "https://api.auricartisan.com/v1/personalize/palettes" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"colors":["#0F172A","#D3AF37"],"styles":["lux","editorial"],"intent":"brand","depth":400,"keep":12}'
```

POST /v1/personalize/system

One brief in, a design system out: the sweep, then from its leading palette, gradients, type pairings, UI colour pairs with WCAG and APCA measured, poster compositions, and a token deck with tonal ramps, light, dark and high-contrast themes and a contrast matrix. **include** trims both the response and the work.

Field	Type	Required	Description
colors , styles , intent	—	No	The brief
include	string array	No	Any of palettes , gradients , typography , ui_pairs , posters , tokens . Default all.
derived	integer	No	Palettes each derived section covers, 1–24. Default 8.
depth	integer	No	Candidates to sample, 8–1200. Default 320.
keep	integer	No	Palettes to keep, 1–120. Default 24.
seed	integer	No	Omit for a random one; it is always returned

Returns **leader**, **palettes**, **gradients**, **typography**, **ui_pairs**, **posters** and **tokens**.

```
curl -X POST "https://api.auricartisan.com/v1/personalize/system" \
-H "x-api-key: $AURIC_API_KEY" \
-H "content-type: application/json" \
-d '{"colors":["#0F172A","#D3AF37"],"styles":["tech","minimal"],"intent":"ui","include":["tokens","ui_pairs"]}'
```

Related pages

- [Palettes, harmonies, gradients and shades](#)
- [Accessibility](#)

SERVICES > PUBLIC API

Usage, jobs, webhooks and GraphQL

Read your usage history and live feed, run long work as asynchronous jobs, subscribe to webhooks, and query the API through GraphQL.

Usage

GET /v1/usage

Per-day, per-endpoint usage history.

Parameter	In	Description
<code>since</code>	query	Earliest day to include, <code>YYYY-MM-DD</code> . Defaults to seven days back.

```
curl "https://api.auricartisan.com/v1/usage" \  
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/usage/me

Your tier, limits and month-to-date consumption.

```
curl "https://api.auricartisan.com/v1/usage/me" \  
-H "x-api-key: $AURIC_API_KEY"
```

```
{"tier":"pro","plan":"specialist","auth_type":"api_key",  
  "limits":{"rate_per_minute":300,"monthly_quota":1000000},  
  "this_month":{"slot":"2026-08","used":412}}
```

`used` counts credits, not calls.

GET /v1/usage/live

The last 100 individual calls, with rolling latency and error statistics. It answers "what is happening right now", where `/v1/usage` answers "how much have I used". Pass the returned `cursor` back as `since` to poll for new calls only.

Parameter	In	Description
<code>limit</code>	query	1–100. Default 50.
<code>since</code>	query	Sequence number from a previous cursor

Returns `cursor`, `stats` and `events`.

GET /v1/usage/stream

The same live feed, pushed over Server-Sent Events (SSE). It emits `hello` on connect with current stats, `usage` for each request, and `heartbeat` every 15 seconds. Connections are capped at 10 minutes; a browser `EventSource` reconnects on its own. 5 credits, charged once when the stream opens; events are free.

```
curl "https://api.auricartisan.com/v1/usage/stream" \  
-H "x-api-key: $AURIC_API_KEY"
```

Async jobs

Long-running work runs as a job: you queue it, get an id back at once, then poll or follow it.

GET /v1/jobs

The job types available, and what each accepts.

POST /v1/jobs/:type

Queue a job. 25 credits. The body is the parameter set for the chosen job type, so its shape varies; **GET /v1/jobs** documents each type.

```
curl -X POST "https://api.auricartisan.com/v1/jobs/palette-batch" \  
-H "x-api-key: $AURIC_API_KEY" \  
-H "content-type: application/json" \  
-d '{"count":500}'
```

GET /v1/jobs/id/:id

Poll one job. 1 credit per poll.

```
curl "https://api.auricartisan.com/v1/jobs/id/<id>" \  
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/jobs/id/:id/stream

Follow a job to completion over SSE instead of polling.

Webhooks

GET /v1/webhooks/events

The event types you can subscribe to. Public, so you can check whether the API fits before you buy. Events mentioned in the documentation include **job.succeeded** and **quota.threshold**, which fires when you cross 80% of your included credits.

```
curl "https://api.auricartisan.com/v1/webhooks/events"
```

POST /v1/webhooks

Subscribe a URL to one or more events. Deliveries are signed: verify the signature header before trusting a payload.

Field	Type	Required	Description
<code>url</code>	string	Yes	HTTPS endpoint to deliver to
<code>events</code>	string array	Yes	From <code>/v1/webhooks/events</code>

```
curl -X POST "https://api.auricartisan.com/v1/webhooks" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"url":"https://example.com/hook","events":["job.succeeded"]}'
```

GET /v1/webhooks, GET /v1/webhooks/:id, DELETE /v1/webhooks/:id

List your subscriptions, fetch one, or revoke one.

```
curl -X DELETE "https://api.auricartisan.com/v1/webhooks/<id>" \
  -H "x-api-key: $AURIC_API_KEY"
```

GraphQL

GET /v1/graphql

The GraphQL playground. Public.

POST /v1/graphql

Run a GraphQL query against the same resolvers as the REST endpoints. 2–4 credits. Useful when one round trip should answer several questions, such as converting a colour, checking its contrast and simulating it at once.

Field	Type	Required	Description
<code>query</code>	string	Yes	The GraphQL document
<code>variables</code>	object	No	Variable values

```
curl -X POST "https://api.auricartisan.com/v1/graphql" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"query":"{ contrast(foreground: \"#000\", background: \"#fff\") { ratio wcag_aa } }"}'
```

Related pages

- [Plans, credits and quotas](#)
- [Reference](#)

SERVICES > PUBLIC API

Vision and Ishihara

Simulate colour-vision deficiencies for a colour, a palette or an image, find pairs that become indistinguishable, and generate Ishihara plates.

The vision endpoints distinguish two kinds of identifier:

- **Models** are simulation engines, such as `brettel-deutan`. List them with `GET /v1/vision/models`.
- **Conditions** are documented vision conditions, such as `deutanopia` or `glaucoma`. List them with `GET /v1/vision/conditions`.

`/v1/vision/simulate` takes a model. The palette and audit endpoints take conditions.

GET /v1/vision/models

The colour-vision deficiency models available.

GET /v1/vision/conditions

Every documented vision condition, and what can simulate it.

GET /v1/vision/conditions/:slug

One condition, with its severity, prevalence and a worked example. `:slug` is a condition slug such as `deutanopia` or `glaucoma`. An unknown slug returns `UNKNOWN_CONDITION`.

```
curl "https://api.auricartisan.com/v1/vision/conditions/deutanopia" \
-H "x-api-key: $AURIC_API_KEY"
```

GET /v1/vision/matrices

The colour-vision deficiency matrices, for applying the transform yourself.

POST /v1/vision/simulate

How one colour appears under a vision deficiency. 1 credit.

Field	Type	Required	Description
<code>color</code>	colour	Yes	The colour
<code>model</code>	string	Yes	A model id such as <code>brettel-deutan</code> ; see /v1/vision/models
<code>severity</code>	number	No	0–1, for anomalous trichromacy

A condition slug such as `deutanopia` is refused here; the error names the model it maps to. To work with conditions, use [POST /v1/vision/palette](#).

```
curl -X POST "https://api.auricartisan.com/v1/vision/simulate" \  
  -H "x-api-key: $AURIC_API_KEY" \  
  -H "content-type: application/json" \  
  -d '{"color":"#c9a227","model":"brettel-deutan","severity":1}'
```

POST /v1/vision/palette

Run a whole palette through many conditions in one call.

Field	Type	Required	Description
<code>colors</code>	array	Yes	Up to 64 colours
<code>conditions</code>	array	No	Condition slugs. Omit for all 17 colour conditions. Spatial conditions are listed under <code>refused</code> .
<code>severity</code>	number	No	0–1, overriding each condition's default
<code>passes</code>	number	No	1–8; repeats the transform

```
curl -X POST "https://api.auricartisan.com/v1/vision/palette" \  
  -H "x-api-key: $AURIC_API_KEY" \  
  -H "content-type: application/json" \  
  -d '{"colors":["#D3AF37","#2E8B57"],"conditions":["deutanopia","tritanopia"]}'
```

POST /v1/vision/audit

Find which pairs in a palette stop being distinguishable under each condition.

Field	Type	Required	Description
<code>colors</code>	array	Yes	At least 2, up to 64

Field	Type	Required	Description
<code>conditions</code>	array	No	Omit for all 17 colour conditions
<code>threshold</code>	number	No	CIEDE2000 difference below which a pair counts as collapsed. Default 5.

```
curl -X POST "https://api.auricartisan.com/v1/vision/audit" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"colors":["#B22222","#228B22"],"conditions":["deuteranopia"]}'
```

POST /v1/vision/simulate-image

Simulate a vision deficiency across a whole image. 10–12 credits. Returns `image/png`.

Field	Type	Required	Description
<code>image</code>	string	Yes	Base64 or a data URL (up to 4 MB)
<code>model</code>	string	Yes	The deficiency to simulate

POST /v1/ishihara/plate

Generate an Ishihara test plate as structured data.

Field	Type	Required	Description
<code>figure</code>	string	Yes	Digits to hide in the plate, for example <code>74</code>
<code>axis</code>	string	No	Deficiency to discriminate: <code>protan</code> , <code>deutan</code> (default) or <code>tritan</code>
<code>size</code>	integer	No	Plate size in pixels, 64–1024. Default 400.
<code>dots</code>	integer	No	Dot count, 50–5000. Default 800.
<code>seed</code>	integer	No	For a repeatable plate

```
curl -X POST "https://api.auricartisan.com/v1/ishihara/plate" \
  -H "x-api-key: $AURIC_API_KEY" \
  -H "content-type: application/json" \
  -d '{"figure":"74","axis":"deutan","size":400}'
```

POST /v1/ishihara/png

Render an Ishihara plate as a PNG. 10–12 credits. Returns `image/png`.

Field	Type	Required	Description
value	string	No	Digits to hide in the plate

Note: Generated plates are for design, teaching and demonstration. They are not a clinical colour-vision test.

Related pages

- [Accessibility](#)
- [Limits and accuracy](#)

SERVICES > PUBLIC API

Other material

Background material for the public API — glossary, limits and accuracy, related products, release notes and privacy.

This folder holds the reference material around the API guides: the terms they use, what the results can and cannot tell you, related products, the documentation's history, and what the API keeps about your use.

Files in this folder

Page	What it covers
Glossary	API, colour-science and accessibility terms
Limits and accuracy	What the results measure, how precisely, and what they are not
Related	Website tools, the dashboard, and other Auric Artisan products
Release notes	Dated changes stated in the API documentation
Privacy	Keys, usage records, what analyzer requests fetch

SERVICES > PUBLIC API

Glossary

Definitions of the API, colour-science and accessibility terms used in the public API documentation.

API terms

API key. A secret string, starting `aa_`, that identifies your account on each request. Created in the dashboard and shown once.

Async job. Work that takes too long for one request. You queue it, receive an id, then poll it or follow it over SSE.

Credit. The unit your monthly quota is counted in. A standard call costs 1; heavier calls cost more.

Cursor. A position in a scanned collection. Pass `next_cursor` back as `cursor` to continue.

Discovery endpoint. A public, unmetered endpoint that describes the API, such as `/v1/catalog`.

GraphQL. A query language that lets one request ask several questions at once, through `/v1/graphql`.

Hard ceiling. The absolute monthly limit, ten times the included allowance, that stops runaway usage.

OpenAPI. A standard, machine-readable description of an HTTP API, served at `/v1/openapi.yaml`.

Overage. Usage beyond the included credits, paid from a prepaid credit balance, and only if you turn it on.

Quota. Your included credits for the month.

Rate limit. How many requests you may make per minute.

Scope and stage. In the analyzer, a stage is one group of checks; **scope** is the list of stages you ask for.

Server-Sent Events (SSE). A way for a server to push a stream of events over one HTTP connection, used by the usage and job streams.

Spend cap. A monthly limit, in money, you set on overage.

Truncated. A scanned-collection response whose scan budget ran out before the page filled; it does not mean nothing matches.

Webhook. A URL you register to receive signed event notifications, such as **job.succeeded**.

Colour science

CCT and Duv. Correlated colour temperature, in kelvin, describes a near-white light by the temperature of the closest black-body colour. Duv is how far the colour sits from that black-body curve (the Planckian locus); beyond about ± 0.05 a temperature no longer describes it.

CIEDE2000, CIE94, CIE76, CMC. Formulae for Delta E, the perceptual difference between two colours. CIEDE2000 is the most accurate and the API's headline figure.

Colour space. A system for describing colours with numbers. The API's registry has 63.

Illuminant. A standard reference light, such as D65 (average daylight), used to define white.

OKLab and OKLCH. Perceptual colour spaces in which equal numeric steps look like equal visual steps. OKLCH writes OKLab as lightness, chroma and hue.

Planckian locus. The curve of colours a black body glows at different temperatures.

White point. The colour a space treats as white, such as D65.

Accessibility

APCA. The Accessible Perceptual Contrast Algorithm from the WCAG 3 drafts, which scores contrast as Lc with polarity.

Auric SD. Auric Artisan's own contrast grading, with Flexible and Strict variants and grades from A+ to F.

Banding. Visible steps in a gradient that should look smooth.

Colour-vision deficiency (CVD). Reduced ability to tell some colours apart, such as protanopia or deuteranopia.

Contrast ratio. WCAG's measure of the difference in luminance between two colours, from 1:1 to 21:1. Body text needs 4.5:1 at level AA.

Polarity. Whether text is dark on light (BoW) or light on dark (WoB).

WCAG. The Web Content Accessibility Guidelines.

SERVICES > PUBLIC API

Limits and accuracy

What the Auric Artisan API's results measure, how precisely, and where they need judgement.

Colour science

- **Delta E:** CIEDE2000 is the headline figure and is verified against the Sharma reference set, the standard test data for that formula.
 - **Colour temperature** is only meaningful near the Planckian locus. The response reports Duv and a `meaningful` flag; beyond about ± 0.05 Duv, a temperature does not describe the colour.
 - **Nearest colour** in the atlas uses CIE76 distance in Lab, and the response names the metric.
-

Accessibility

- Contrast checks work on the colours you send. They cannot know what is actually behind text on a real page; for that, measure the rendered page, for example with the website's Analyzer or the browser extension.
 - APCA is a draft method for WCAG 3; its thresholds may change.
 - `/v1/accessibility/map` returns `null` lightness for hues that can never reach the target on a background. That is a real answer, not a failure.
 - `/v1/accessibility/audit` and `/v1/accessibility/picks` refuse to return colours that do not meet the target.
-

Vision simulation

- Simulations model how colours may appear with a deficiency; they are not a diagnosis, and individual vision varies.
 - `/v1/vision/palette` and `/v1/vision/audit` simulate colour conditions; spatial conditions are listed under `refused`.
-

Collections

- Library and corpus records are derived from a published seed, so ids are reproducible and citable.
- Statistics endpoints count or sample the collection and state the sample they used.

- In the five-million-record collections, a filtered browse scans a budget of records. A **truncated** page is not proof that nothing matches.

Analyzer

- **/v1/analyzer/inspect** reads page source only. It refuses contrast, the accessibility engine, deep DOM analysis, performance, the rendered-versus-raw comparison and simulation rather than guess.
- **palette** returns declared colours, not the colours that cover the most pixels; **media** misses anything added by JavaScript.
- There is no overall score. The markup-level **a11y** category is not the analyzer's full accessibility engine.
- PDF reports show Latin characters only.

Operational limits

- Image request bodies are limited to 4 MB.
- Usage streams are capped at 10 minutes per connection.
- Rate limits, quotas and ceilings are listed in [Plans, credits and quotas](#).

Related pages

- [Reference](#)
- [Analyzer](#)

SERVICES > PUBLIC API

Privacy

What the Auric Artisan API keeps about your keys and usage, and what analyzer requests fetch, as its documentation describes.

This page covers what the API's documentation says about data. The site's privacy policy, <https://auricartisan.com/legal/privacy/>, covers your account as a whole.

Keys

- The full key is shown once, when you create it, and stored only as a SHA-256 hash. Nobody can recover it later.
- Each successful request updates the key's "last used" time, visible in your dashboard.

Usage records

The API keeps a record of your use so it can meter and report it:

- per-day, per-endpoint history, available through `GET /v1/usage` ;
- the last 100 individual calls with latency and errors, through `GET /v1/usage/live` and the stream;
- your month-to-date credits, through `GET /v1/usage/me` .

Error responses carry a `request_id` so a specific call can be traced when you ask for support.

What you send

The API processes the colours, palettes, images, reports and other values you send in order to answer the request. Do not send personal data you do not need to.

Analyzer requests

`POST /v1/analyzer/inspect` fetches the public URL you name and reads its source. Private and reserved addresses are refused. `POST /v1/analyzer/render` fetches nothing: it renders the report you send.

Webhooks

Webhook deliveries go only to the HTTPS URLs you register, and are signed so you can verify they came from the API.

Related pages

- [Authentication](#)
- [Plans, credits and quotas](#)

SERVICES > PUBLIC API

Release notes

The dated changes to the public API that its documentation states.

The API has no separate public changelog. These are the dated facts its documentation states. The current version is always reported by `GET /v1` .

3 September 2026 — Analyzer API guide

- A guide to the three analyzer endpoints was published: `GET /v1/analyzer/stages` , `POST /v1/analyzer/inspect` and `POST /v1/analyzer/render` .
- The analyzer endpoints read page source only, refuse checks that need a browser, and return no overall score.
- Costs: 1, 12 and 10 credits.

12 August 2026 — Credits, overage and new Industrial Pro allowance

- The REST API reference was updated.
- The monthly quota is counted in credits, with heavier endpoints costing more than one credit.
- Prepaid, opt-in pay-as-you-go overage, a monthly spend cap and a hard ceiling of ten times the included allowance.
- Industrial Pro's allowance became 10,000,000 credits a month for subscriptions started on or after this date. Earlier Industrial Pro subscriptions, sold as "Unlimited", keep 1,000,000,000 credits a month.

8 July 2026 — REST API reference

- The REST API reference was published, with API keys created in the account dashboard.

Upcoming

The analyzer guide lists two things "not here yet": a rendered scan (contrast, the accessibility engine and deep DOM comparison through the API) and a whole-site crawl as a job.

SERVICES > PUBLIC API

Related

The dashboard, website tools and other Auric Artisan products that relate to the public API.

Your account

- **Dashboard:** create and revoke keys, watch usage and cost, buy credits and set spend controls at <https://auricartisan.com/auth/dashboard/>. See the [Account and billing](#) docs.
- **Plans:** compare plans at <https://auricartisan.com/company/pricing/>.
- **Billing policy:** <https://auricartisan.com/legal/subscription-billing/>.

Documentation on the site

- REST API reference: <https://auricartisan.com/library/documentation/api/>
- Analyzer API guide: <https://auricartisan.com/library/documentation/api/analyzer/>

Website tools that share the API's engines

Tool	Docs
Analyzer	Analyzer

Tool	Docs
Contrast Checker	Contrast Checker
Vision Simulation	Vision Simulation
Ishihara Plate Generator	Ishihara plates
Personalisation Generator	Personalisation Generator
Colour libraries	Colour libraries

Other products

- [Browser extension](#): the same colour and accessibility checks on any page, without an account.
- [Font CDN](#) and [Icon CDN](#).